

Platform Engineering Handbook

From Fundamentals to Production-Grade Azure Lab

Gopal Khandale

2026

Contents

1	Preface	8
2	Part I — Foundations	9
3	Chapter 1: The Cloud and Why We’re Here	10
3.1	1.1 What “the cloud” actually is	10
3.2	1.2 Three service models	10
3.3	1.3 The three big clouds	10
3.4	1.4 Regions, availability zones, and resource groups	11
	3.4.1 Regions	11
	3.4.2 Availability Zones (AZ)	11
	3.4.3 Resource Groups	11
3.5	1.5 Cost model	11
3.6	1.6 Why this matters in interviews	12
3.7	1.7 Exercises	12
4	Chapter 2: Linux for Platform Engineers	13
4.1	2.1 The shell	13
4.2	2.2 Essential commands	13
	4.2.1 File navigation	13
	4.2.2 File manipulation	13
	4.2.3 Text processing	14
	4.2.4 Processes	14
	4.2.5 File transfer	14
4.3	2.3 Permissions	14
4.4	2.4 SSH and remote work	15
	4.4.1 Generate a key pair	15
	4.4.2 SSH config file	15
	4.4.3 LocalForward (tunneling)	15
4.5	2.5 Environment variables	15
4.6	2.6 systemd (process supervision)	16
4.7	2.7 Exercises	16
5	Chapter 3: Git and Version Control	17
5.1	3.1 What Git actually is	17
5.2	3.2 The three states of a file	17
5.3	3.3 Daily commands	17

5.4	3.4 Branching strategies	18
5.4.1	Git Flow (heavy)	18
5.4.2	GitHub Flow (modern, recommended)	18
5.4.3	Trunk-Based Development	18
5.5	3.5 Pull Requests (PRs)	19
5.6	3.6 Merge conflicts	19
5.7	3.7 Rewriting history (carefully)	19
5.8	3.8 .gitignore	19
5.9	3.9 Git in CI/CD	20
5.10	3.10 Why this matters in interviews	20
5.11	3.11 Exercises	20
5.11.1	Visual: Git commit graph	20
5.11.2	Visual: PR workflow	21
6	Chapter 4: Networking Essentials	22
6.1	4.1 The OSI model (simplified)	22
6.2	4.2 IP addresses and CIDR	22
6.3	4.3 Public vs Private IPs	23
6.4	4.4 DNS	23
6.5	4.5 TCP/UDP and ports	23
6.6	4.6 Firewalls and NSGs	24
6.6.1	Default NSG rules (Azure)	24
6.6.2	Visual: NSG rule evaluation	24
6.7	4.7 Load Balancers	24
6.8	4.8 TLS/HTTPS	25
6.9	4.9 Why this matters in interviews	25
6.10	4.10 Exercises	25
7	Chapter 5: Containers and Docker	27
7.1	5.1 What problem do containers solve?	27
7.2	5.2 VMs vs Containers	27
7.3	5.3 How containers work (the technical bits)	28
7.4	5.4 Docker concepts	28
7.5	5.5 Dockerfile anatomy	28
7.6	5.6 Common Docker commands	29
7.7	5.7 Image layers and caching	29
7.8	5.8 Images and tags	29
7.9	5.9 Container registries	30
7.10	5.10 Security best practices	30
7.11	5.11 Why this matters in interviews	31
7.12	5.12 Exercises	31
8	Chapter 6: Kubernetes Deep Dive	32
8.1	6.1 What Kubernetes is	32
8.2	6.2 The architecture	32
8.3	6.3 Core objects	33
8.3.1	Pod	33
8.3.2	Deployment	33

8.3.3	Service	34
8.3.4	ConfigMap and Secret	34
8.3.5	Namespace	35
8.4	6.4 kubectl daily commands	35
8.5	6.5 Pod lifecycle	36
8.6	6.6 Pod scheduling	36
8.7	6.7 Networking inside Kubernetes	37
8.8	6.8 StatefulSet vs Deployment	37
8.9	6.9 PersistentVolume and PersistentVolumeClaim	38
8.10	6.10 RBAC (Role-Based Access Control)	38
8.11	6.11 Common pod failure modes	39
8.12	6.12 Why this matters in interviews	39
8.13	6.13 Exercises	39
9	Chapter 7: Infrastructure as Code with Terraform	41
9.1	7.1 What is IaC?	41
9.2	7.2 Terraform basics	41
9.3	7.3 Terraform workflow	42
9.4	7.4 State	42
9.4.1	Local vs Remote state	43
9.4.2	The chicken-and-egg problem	43
9.5	7.5 Variables and outputs	43
9.6	7.6 Modules	44
9.7	7.7 State locking	45
9.8	7.8 Data sources	45
9.9	7.9 Common gotchas	45
9.10	7.10 Production patterns	46
9.11	7.11 Why this matters in interviews	46
9.12	7.12 Exercises	46
10	Chapter 8: Helm — Package Manager for Kubernetes	47
10.1	8.1 The problem Helm solves	47
10.2	8.2 Chart structure	47
10.3	8.3 Templates with values	47
10.4	8.4 Common Helm commands	48
10.5	8.5 Subcharts / umbrella charts	49
10.6	8.6 Conditionals and loops	50
10.7	8.7 Helm hooks	50
10.8	8.8 Helm vs Kustomize	50
10.9	8.9 Why this matters in interviews	51
10.10	8.10 Exercises	51
11	Chapter 9: Azure Specifics for Platform Engineers	52
11.1	9.1 Azure resource hierarchy	52
11.2	9.2 Authentication models	52
11.2.1	Azure CLI login (interactive)	52
11.2.2	Service Principal (for automation)	52
11.2.3	Workload Identity (modern, no secrets)	53

11.3	9.3 RBAC roles	53
11.4	9.4 Key Azure services for platform engineers	53
11.4.1	AKS (Azure Kubernetes Service)	53
11.4.2	ACR (Azure Container Registry)	54
11.4.3	Virtual Network (VNet)	54
11.4.4	Log Analytics Workspace	55
11.4.5	Storage Account	55
11.5	9.5 Free Trial constraints	55
11.6	9.6 Common az CLI commands	55
11.7	9.7 AKS networking modes	56
11.8	9.8 Why this matters in interviews	56
11.9	9.9 Exercises	56
12	Part II — Building the Lab	57
13	Chapter 10: Lab Architecture Overview	58
13.1	10.1 What we’re building	58
13.2	10.2 The 6 iterations	59
13.3	10.3 The lab tells three CI stories	59
13.4	10.4 Why this architecture	60
14	Chapter 11: Iteration 1 — Bootstrap and Network	61
14.1	11.1 The chicken-and-egg fix	61
14.1.1	Bootstrap workflow	61
14.2	11.2 The <code>bootstrap/main.tf</code>	62
14.3	11.3 The network module	63
14.4	11.4 NSGs — the explicit deny gotcha	63
14.4.1	Why this matters	64
14.5	11.5 Outputs	64
14.6	11.6 Apply iter 1	65
15	Chapter 12: Iteration 2 — ACR and AKS	66
15.1	12.1 ACR module	66
15.2	12.2 AKS module	66
15.2.1	The RBAC gotcha	68
15.3	12.3 Apply iter 2	68
15.4	12.4 Get cluster access	68
16	Chapter 13: Iteration 3 — Platform Components	69
16.1	13.1 Istio	69
16.1.1	Why a service mesh?	69
16.1.2	The Istio architecture	69
16.1.3	Install Istio (Helm)	70
16.1.4	Gateway and VirtualService	70
16.1.5	Sidecar tuning (the resource gotcha)	71
16.2	13.2 ArgoCD	71
16.2.1	Why GitOps?	71
16.2.2	Install ArgoCD	72

16.2.3	Application CRD	72
16.3	13.3 Jenkins	72
16.3.1	Why Jenkins (still)?	72
16.3.2	Install Jenkins (Helm)	72
16.3.3	Kubernetes plugin for dynamic agents	73
17	Chapter 14: Iteration 4 — Three-Tier App	75
17.1	14.1 The umbrella chart	75
17.2	14.2 Frontend subchart	75
17.3	14.3 Database subchart (StatefulSet)	76
17.4	14.4 Deploy	77
17.5	14.5 Test via port-forward	78
18	Chapter 15: Iteration 5 — Real Code + Jenkins Pipeline	79
18.1	15.1 Backend app (Node + Express)	79
18.2	15.2 Backend Dockerfile	80
18.3	15.3 Jenkinsfile — multi-container pipeline	80
18.4	15.4 What this pipeline does	83
18.5	15.5 The ACR secret	83
18.6	15.6 ArgoCD applications (drift detection)	83
19	Chapter 16: Iteration 6 — GitHub Actions + Argo Rollouts + Prometheus	85
19.1	16.1 Argo Rollouts	85
19.1.1	What it is	85
19.1.2	Install	86
19.1.3	Rollout spec	86
19.1.4	Commands	86
19.2	16.2 Prometheus	87
19.2.1	What it is	87
19.2.2	Install via kube-prometheus-stack	87
19.2.3	PodMonitor (CRD that creates scrape config)	87
19.2.4	PromQL basics	88
19.3	16.3 AnalysisTemplate	88
19.3.1	Two-gate analysis explained	89
19.3.2	Flow	90
19.4	16.4 GitHub Actions workflow	90
19.5	16.5 What makes this production-grade	93
19.6	16.6 The full flow visualized	94
20	Chapter 17: Iteration 7 — CircleCI (the third CI for portfolio)	96
20.1	17.1 Why a third CI?	96
20.2	17.2 CircleCI mental model	96
20.3	17.3 CircleCI configuration	96
20.4	17.4 CircleCI-specific concepts	99
20.4.1	Executors	99
20.4.2	Orbs	99
20.4.3	Contexts	100
20.4.4	Test parallelism	100

20.4.5	Workspaces (sharing files between jobs)	100
20.4.6	Manual approval	100
20.5	17.5 CircleCI vs GitHub Actions comparison	101
20.6	17.6 Common gotchas (heredoc, etc.)	101
21	Part III — Advanced Topics	102
22	Chapter 18: Observability	103
22.1	18.1 The three pillars	103
22.2	18.2 Metrics with Prometheus	103
22.2.1	What we measure	103
22.2.2	Exposition format	104
22.2.3	Cardinality matters	104
22.3	18.3 Logs	104
22.3.1	Structured logging	104
22.4	18.4 Traces	105
22.5	18.5 SLI, SLO, SLA	105
22.6	18.6 Alerting	105
22.6.1	Alert hygiene	106
23	Chapter 19: Security	107
23.1	19.1 Defense in depth	107
23.2	19.2 Secrets management	108
23.2.1	Bad	108
23.2.2	OK	108
23.2.3	Best	108
23.3	19.3 Container security	108
23.4	19.4 Network policies	108
23.5	19.5 Pod Security Standards (PSS)	109
23.6	19.6 RBAC principles	109
24	Chapter 20: Production-Grade Gaps in Our Lab	110
24.1	20.1 Identity & Secrets	110
24.2	20.2 Network	110
24.3	20.3 Supply chain	110
24.4	20.4 Reliability	111
24.5	20.5 Process	111
25	Chapter 21: Interview Questions and Answers	112
25.1	Kubernetes	112
25.1.1	1. Difference between a Deployment and a StatefulSet?	112
25.1.2	2. Walk through what happens when you run <code>kubectl apply -f deployment.yaml</code>	112
25.1.3	3. How does a Service load balance to multiple Pods?	113
25.1.4	4. Pod stuck in Pending. Debug steps.	113
25.1.5	5. What's the role of a sidecar container?	113
25.2	CI/CD	113
25.2.1	6. GitOps vs CIOps?	113

25.2.2	7. Blue/green vs canary?	114
25.2.3	8. How do you handle database migrations during deploys?	114
25.2.4	9. How do you decide between Helm and Kustomize?	114
25.2.5	10. Walk through a CI failure on a flaky test.	114
25.3	Terraform	114
25.3.1	11. What's a state file? Why remote backend?	114
25.3.2	12. What does <code>terraform plan</code> actually do?	114
25.3.3	13. Two devs run <code>terraform apply</code> simultaneously.	115
25.3.4	14. How do you import existing cloud resources into Terraform?	115
25.4	Azure	115
25.4.1	15. AKS vs EKS vs GKE — when do you pick each?	115
25.4.2	16. Why kubenet vs Azure CNI?	115
25.4.3	17. Service Principal vs Managed Identity?	115
25.5	Observability	116
25.5.1	18. RED method vs USE method?	116
25.5.2	19. What's an SLO?	116
25.5.3	20. How does Prometheus collect metrics?	116
25.6	Argo / GitOps specific	116
25.6.1	21. How does Argo Rollouts work under the hood?	116
25.6.2	22. Why might <code>prePromotionAnalysis</code> falsely pass?	116
25.6.3	23. How does ArgoCD detect drift?	117
25.6.4	24. ArgoCD shows Suspended. Is that bad?	117
26	Part IV — Appendix	118
27	Appendix A: Quick Reference Cheatsheets	119
27.1	Kubernetes daily commands	119
27.2	Helm commands	120
27.3	Terraform commands	120
27.4	Argo Rollouts commands	121
27.5	Azure CLI	121
27.6	Docker	121
27.7	Git	121
28	Appendix B: Building this PDF/DOCX	123
28.1	To PDF	123
28.2	To DOCX	123
28.3	Quick install on Ubuntu	123
29	Appendix C: Where to Go Next	125
29.1	Advanced Kubernetes	125
29.2	Advanced CI/CD	125
29.3	Reliability	125
29.4	Security	125
29.5	Cost	126
30	End	127

Chapter 1

Preface

This handbook is built around a real working lab on Azure — an AKS cluster with a three-tier app, two CI/CD pipelines, GitOps, progressive delivery, and observability. Every concept is grounded in something you can run.

Read top to bottom for a complete journey from “what is the cloud” to “blue/green deployment with Prometheus gates.” Skip to specific chapters if you need to fill a knowledge gap.

The lab repo is at <https://github.com/gkhandale-aziro/azure-platform-lab>.

How to use this book:

1. Read each chapter
2. Run the commands on your machine
3. Read the “Why this matters” callouts — they’re the interview answers
4. Do the exercises at the end of each chapter

Chapter 2

Part I — Foundations

Chapter 3

Chapter 1: The Cloud and Why We're Here

3.1 1.1 What “the cloud” actually is

The cloud is just other people’s computers. Specifically, it’s:

- Massive data centers (each holds 100,000+ physical servers)
- High-speed networking between racks
- Software that lets you **rent** parts of those servers by the hour
- An API to provision, configure, and destroy resources on demand

Before the cloud, to run a website you bought a physical server, drove it to a data center, paid for rack space, installed an OS, and dealt with everything yourself. The cloud abstracts this: you make an API call and 60 seconds later you have a Linux VM.

3.2 1.2 Three service models

Model	What you manage	What the cloud manages	Example
IaaS (Infrastructure as a Service)	OS, runtime, app	Hardware, networking, virtualization	Azure VM, AWS EC2
PaaS (Platform as a Service)	App + config	Everything below the app	Azure App Service, Heroku
SaaS (Software as a Service)	Your data only	The entire stack	Office 365, Salesforce

Where Kubernetes fits: Kubernetes itself is between IaaS and PaaS. You manage app config + scaling rules; the cluster manages where containers run.

3.3 1.3 The three big clouds

	Azure	AWS	Google Cloud (GCP)
Owner	Microsoft	Amazon	Google
Market share (2026)	~24%	~30%	~12%
Strong in	Enterprises, Windows shops, .NET, hybrid cloud	Pure size, mature service catalog	Data/ML, Kubernetes (they invented it)
Compute	Azure Compute, AKS	EC2, EKS	Compute Engine, GKE
Object storage	Azure Blob	S3	Cloud Storage
Managed K8s	AKS	EKS	GKE
Container registry	ACR	ECR	Artifact Registry
Identity	Microsoft Entra ID (formerly Azure AD)	IAM	Cloud IAM

For this book we use Azure. All concepts transfer; only the resource names change.

3.4 1.4 Regions, availability zones, and resource groups

3.4.1 Regions

A geographic area with one or more data centers. Examples: `eastus2`, `westeurope`, `centralindia`.

Pick a region close to your users for low latency. Pick a region with your required services (not all services are in all regions — e.g., some GPU SKUs are only in `eastus`).

3.4.2 Availability Zones (AZ)

Within a region, separate physical locations with independent power/cooling/network. Typically 3 AZs per region (some have 2, some have none).

Why this matters: a single rack/data center can fail. Spread workloads across AZs for high availability.

3.4.3 Resource Groups

A logical container for related Azure resources. Everything in a resource group can be deleted with one command (`az group delete`).

Rule of thumb: one resource group per environment-or-app. Our lab uses `gskplat-rg-platform` for the AKS cluster + ACR.

3.5 1.5 Cost model

Cloud is rented. You pay per hour (or per second). Common cost dimensions:

Resource	Cost driver
VMs	vCPU + RAM × hours + disk + outbound bandwidth
Storage	GB × month + transactions
Networking	Outbound bandwidth (egress) + inter-region traffic
Managed services (AKS, ACR)	Usually free control plane + you pay for underlying VMs

Lab cost story: Our 2-node D2s_v3 cluster runs ~\$140/month at 24/7. By `az aks stop`-ing nights/weekends, we get to ~\$35/month. Control plane is free in AKS.

3.6 1.6 Why this matters in interviews

“Why did you choose Azure for this project?”

Good answer: > “I needed to learn Azure for the role’s tech stack. The Free Trial gave me \$200 of credit, which lasted ~30 days running an AKS cluster part-time. The lab pattern transfers to AWS/GCP — same concepts (managed K8s + container registry + service mesh + GitOps), different SKUs.”

Bad answer: > “It was free.”

The good answer shows you understand portability of skills.

3.7 1.7 Exercises

1. Create a free Azure account (<https://azure.microsoft.com/free>)
2. Run `az login` and identify your subscription ID
3. Create a resource group: `az group create -n test-rg -l eastus2`
4. List it: `az group list --output table`
5. Delete it: `az group delete -n test-rg --yes`

Chapter 4

Chapter 2: Linux for Platform Engineers

You will live in Linux terminals. Master these basics or struggle forever.

4.1 2.1 The shell

Most Linux systems use **bash** or **zsh**. They interpret commands and have features like:

- **Pipes** (`|`) — output of one command into another: `ls | grep test`
- **Redirection** (`>`, `>>`) — write to file: `ls > files.txt`
- **Background processes** (`&`) — run in background: `kubectl port-forward ... &`
- **Variables** (`$VAR`) — store values: `NAME=alice; echo $NAME`

4.2 2.2 Essential commands

4.2.1 File navigation

Command	What
<code>ls -la</code>	list files with details
<code>cd /path</code>	change directory
<code>pwd</code>	print working directory
<code>tree</code>	recursive directory view (if installed)
<code>find . -name "*.yaml"</code>	search for files

4.2.2 File manipulation

Command	What
<code>cat file.txt</code>	print whole file
<code>head -20 file.txt</code>	first 20 lines
<code>tail -20 file.txt</code>	last 20 lines
<code>tail -f file.log</code>	follow new log lines (useful for live logs)

Command	What
<code>less file.txt</code>	paginated viewer (q to quit)
<code>grep "error" file.log</code>	search lines containing “error”
<code>grep -r "pattern" .</code>	recursive grep
<code>wc -l file.txt</code>	count lines

4.2.3 Text processing

Command	What
<code>awk '{print \$1}' file</code>	print first column
<code>sed 's/old/new/g' file</code>	substitute
<code>cut -d',' -f2 file.csv</code>	extract column 2 from CSV
<code>sort / uniq -c</code>	sort and count duplicates
<code>jq '.field'</code>	parse JSON (<code>apt install jq</code>)

4.2.4 Processes

Command	What
<code>ps aux</code>	list all processes
<code>pgrep -f "pattern"</code>	find PIDs matching pattern
<code>kill -9 PID</code>	force-kill process
<code>top / htop</code>	live process viewer
<code>ss -tlnp</code>	listening TCP ports
<code>lsof -i :8080</code>	what process owns port 8080

4.2.5 File transfer

Command	What
<code>scp file user@host:/path</code>	copy file to remote host
<code>rsync -avz src/ user@host:/dst/</code>	sync directory
<code>curl -O https://example.com/file</code>	download
<code>wget https://example.com/file</code>	download (alternative)

4.3 2.3 Permissions

Linux file permissions look like `-rwxr-xr-x`:

```

- rwx r-x r-x
↑  ↑  ↑  ↑
  others can read+execute
  group can read+execute
  owner can read+write+execute
file type (- = file, d = directory)

```

Common commands: - `chmod 755 file` — set permissions (7=rwx, 5=r-x, 4=r-) - `chmod +x script.sh` — make executable - `chown user:group file` — change ownership - `sudo` — run as root

4.4 2.4 SSH and remote work

4.4.1 Generate a key pair

```
ssh-keygen -t ed25519 -C "your@email.com"
# Creates ~/.ssh/id_ed25519 (private) and id_ed25519.pub (public)
```

Never share the private key. Put the public key on remote servers in `~/.ssh/authorized_keys`.

4.4.2 SSH config file

At `~/.ssh/config`:

```
Host azurelab
  HostName 172.30.44.145
  User aziro
  ServerAliveInterval 60
  LocalForward 59080 localhost:8080
```

Then just `ssh azurelab` instead of typing the whole command.

4.4.3 LocalForward (tunneling)

The `LocalForward 59080 localhost:8080` line means: traffic to `localhost:59080` on your laptop goes through the SSH tunnel and hits `localhost:8080` on the remote host. Critical for accessing services inside firewalled environments.

We use this in the lab to access Jenkins/Grafana/ArgoCD running on the AKS cluster from a laptop browser.

4.5 2.5 Environment variables

```
export MY_VAR=value           # set for current shell + children
echo $MY_VAR                  # print
echo "${MY_VAR}_suffix"       # use in strings
unset MY_VAR                   # remove
env                             # list all env vars
```

Common gotcha: `export` makes the variable available to child processes. Without `export`, only the current shell can see it.

To persist across logins, add to `~/.bashrc` (bash) or `~/.zshrc` (zsh):

```
# ~/.bashrc
export AZURE_SUBSCRIPTION="my-sub-id"
```

Our lab uses `~/.azure-lab.env` sourced from `~/.bashrc` so `ARM_*` env vars persist.

4.6 2.6 systemd (process supervision)

systemd manages long-running services on modern Linux. Key commands:

```
systemctl status nginx      # is nginx running?
sudo systemctl start nginx  # start
sudo systemctl stop nginx
sudo systemctl restart nginx
sudo systemctl enable nginx # start at boot
sudo systemctl disable nginx # don't start at boot
journalctl -u nginx         # service logs
journalctl -fu nginx        # follow service logs
```

You won't deploy systemd services in Kubernetes (containers handle that), but you'll diagnose them on VMs, build servers, etc.

4.7 2.7 Exercises

1. SSH into a VM (AWS Lightsail or Azure VM, both have free tiers)
 2. Create an SSH config alias
 3. Set up an SSH tunnel from your laptop to a service running on the VM
 4. Use `tail -f` on a log file while doing something that updates it
 5. Use `grep` + `awk` to extract specific data from a multiline file
-

Chapter 3: Git and Version Control

5.1 3.1 What Git actually is

Commits form a directed acyclic graph (DAG). A **branch** is a moveable pointer to a commit. **HEAD** is a pointer to the current commit.

```
Working directory    git add    Staging area    git commit    Repository
                    ↑
                    git checkout
```

5.3 3.3 Daily commands

```
# Daily flow
git status # what's changed?
```

```

git diff                                # show changes (unstaged)
git diff --staged                       # show staged changes
git add file.txt                       # stage one file
git add .                              # stage everything
git commit -m "fix: typo"              # commit staged changes
git log --oneline -20                  # recent commits

# Branches
git branch                             # list branches
git branch feature/new-thing           # create branch (doesn't switch to it)
git checkout feature/new-thing         # switch to branch
git checkout -b feature/new-thing      # create + switch
git checkout master                    # switch back
git merge feature/new-thing            # merge a branch into current
git branch -d feature/new-thing        # delete branch

# Remote work
git remote -v                          # list remotes
git fetch                              # download remote refs (no merge)
git pull                               # fetch + merge
git pull --rebase                      # fetch + rebase (cleaner history)
git push                               # upload current branch
git push origin master                 # explicit remote + branch

```

5.4 3.4 Branching strategies

5.4.1 Git Flow (heavy)

- master = production
- develop = next release
- feature/* = work branches
- release/* = stabilization
- hotfix/* = emergency fixes

Old-fashioned, rarely used now. Too many long-lived branches.

5.4.2 GitHub Flow (modern, recommended)

- master (or main) = always deployable
- feature/* = short-lived branches
- Open a PR → review → merge → delete branch

This is what most modern teams use. Our lab uses this.

5.4.3 Trunk-Based Development

- Everyone commits to master/trunk directly
- Heavy automated testing
- Feature flags hide incomplete features

Used by Google, Facebook. Requires mature CI/CD.

5.5 3.5 Pull Requests (PRs)

A PR is a request to merge changes from one branch into another. The PR page: - Shows the diff - Runs CI (tests, lint, scans) - Has a discussion thread - Requires approvals (configurable)

Why PRs matter: They're the gate between work and production. Reviews catch bugs. CI catches regressions. They create an audit trail.

5.6 3.6 Merge conflicts

When two branches changed the same line, git can't auto-merge. Resolution:

```
git pull --rebase origin master
# CONFLICT in file.txt

# Edit file.txt - find conflict markers:
# <<<<<< HEAD
# your version
# =====
# their version
# >>>>>> commit-sha

# Pick one, save the file
git add file.txt
git rebase --continue
```

You'll hit this constantly when multiple people work on the same repo. Our lab hit this many times when GitHub Actions' bot commits conflicted with local commits.

5.7 3.7 Rewriting history (carefully)

```
git commit --amend           # change last commit message or add files
git reset HEAD~1             # undo last commit, keep changes staged
git reset --hard HEAD~1      # undo last commit, lose changes
git rebase -i HEAD~5         # interactive rebase: squash, reorder, edit
git revert <commit>          # create new commit that undoes <commit>
```

Rule: Never rewrite history that's been pushed to a shared branch. Use **revert** for shared work, **reset/rebase** only for local branches.

5.8 3.8 .gitignore

Tells Git which files to ignore:

```
# Common patterns
node_modules/
*.log
```

```
.env
.terraform/
*.tfstate
*.tfstate.backup
```

Critical for keeping secrets, build artifacts, and machine-specific files out of the repo. Our lab gitignores `sp-terraform.json` (SP credentials).

5.9 3.9 Git in CI/CD

CI systems do roughly this:

```
git clone <repo>
git checkout <branch-or-sha>
# run tests, build, deploy
```

The `<sha>` is the immutable identifier. We use the **short SHA (7 chars)** as our image tag in the lab — guarantees the image matches a specific commit.

5.10 3.10 Why this matters in interviews

“How do you roll back a bad deploy?”

Good answer: > “Three layers. Inside the rolling window — Argo Rollouts has the previous ReplicaSet alive for 30 seconds, so `kubectl argo rollouts undo` is instant. Beyond that, `git revert <bump-commit>` and let ArgoCD reconcile — image tags are git SHAs, so reverting the commit means deploying the previous image. Longer term, every deploy is a git commit, so the audit trail makes it easy to identify the bad change and learn from it.”

5.11 3.11 Exercises

1. Create a private repo on GitHub
2. Clone it, make a commit, push it
3. Create a branch, change something, push the branch
4. Open a PR on GitHub, review it, merge it
5. Practice resolving a merge conflict (create one intentionally)
6. Use `git log --oneline --graph --all` to visualize branch history
7. Try `git rebase -i HEAD~3` to squash 3 commits into 1

5.11.1 Visual: Git commit graph

```
master          A B C          G H
                \              /
feature/foo      D E F  /
                \
feature/bar      X Y
```

A = initial commit
B = added README

C = bug fix on master
D-F = feature/foo work
G = merged feature/foo into master
H = latest commit
X-Y = work on feature/bar (not yet merged)

5.11.2 Visual: PR workflow

Developer
local branch
feature/new-x

`git push origin feature/new-x`

GitHub
feature/new-x branch
Open Pull Request

Automated checks (CI)

- Unit tests
- Linter
- Security scan
- Build

Human review

- Approve / Request changes
- Comments on specific lines

Merge to master
Branch deleted
Deploy pipeline triggered

Chapter 6

Chapter 4: Networking Essentials

You can't operate distributed systems without understanding networking. Master these concepts.

6.1 4.1 The OSI model (simplified)

Layer 7: Application	(HTTP, gRPC, SSH)	← Where you code
Layer 6: Presentation	(TLS encryption)	
Layer 5: Session		
Layer 4: Transport	(TCP, UDP)	← Ports live here
Layer 3: Network	(IP, routing)	← IP addresses
Layer 2: Data Link	(Ethernet)	← MAC addresses
Layer 1: Physical	(cables, radio)	

In practice, you'll talk about **L4** (TCP load balancers) and **L7** (HTTP proxies, ingress controllers).

6.2 4.2 IP addresses and CIDR

An IPv4 address is 32 bits: 10.0.1.15 = four octets.

CIDR notation says how many bits are the network prefix: - 10.0.0.0/16 — first 16 bits fixed, last 16 bits available = 65,536 IPs - 10.0.1.0/24 — first 24 bits fixed = 256 IPs - 10.0.1.0/27 — first 27 bits fixed = 32 IPs

CIDR	Bits	Addresses	Use case
/8	24	16,777,216	Internet ranges
/16	16	65,536	VNet

/24	8	256	Subnet
/27	5	32	Small subnet (mgmt)
/32	0	1	Single host

Our lab VNet:

gskplat-vnet-shared	10.0.0.0/16	65,536 IPs total
snet-aks	10.0.1.0/24	256 IPs (cluster nodes)
snet-apps	10.0.2.0/24	256 IPs (future apps)
snet-mgmt	10.0.3.0/27	32 IPs (jumpbox/mgmt)

6.3 4.3 Public vs Private IPs

Private IP ranges (RFC 1918):

10.0.0.0/8	- most common in cloud VNets
172.16.0.0/12	- Docker default
192.168.0.0/16	- home routers

Public IPs:

Everything else. Globally routable on the internet.

VMs in a VNet get private IPs. To reach the internet, they need either: - A **NAT gateway** (translates private→public outbound) - A **public IP** attached directly - Through a **load balancer**

6.4 4.4 DNS

DNS maps names to IPs:

api.example.com resolved to 20.94.18.66

Types of DNS records: | Type | Maps to | Example | |---|---| | A | IPv4 | example.com → 1.2.3.4 | | AAAA | IPv6 | example.com → ::1 | | CNAME | Another domain | www.example.com → example.com | | MX | Mail server | example.com → mail.example.com | | TXT | Text (used for verification) | _acme-challenge.example.com → "abc123" |

Inside Kubernetes: CoreDNS resolves service names. backend.dev.svc.cluster.local resolves to the backend Service's ClusterIP.

6.5 4.5 TCP/UDP and ports

A **port** is a number (0-65535) that identifies which application on a host gets the traffic.

Connection format: <ip>:<port>

Web server	:	80 (HTTP), 443 (HTTPS)
SSH	:	22
PostgreSQL	:	5432
Kubernetes API	:	6443
Our backend	:	5678

TCP (Transmission Control Protocol): - Reliable, ordered, connection-oriented - 3-way handshake (SYN → SYN-ACK → ACK) - Used for HTTP, SSH, databases

UDP (User Datagram Protocol): - Best-effort, no connection - Used for DNS queries, streaming, real-time games

6.6 4.6 Firewalls and NSGs

A firewall filters traffic based on rules. Rules typically have: - Source IP/range - Destination IP/range - Port - Protocol (TCP/UDP) - Action (Allow/Deny) - Priority

Azure's **Network Security Group (NSG)** is a stateful firewall attached to subnets or NICs.

6.6.1 Default NSG rules (Azure)

Priority	Name	Source	Dest	Port	Action
65000	AllowVnetInBound	VirtualNet	*	*	Allow
65001	AllowAzureLoadBalancerInBound	AzureLB	*	*	Allow
65500	DenyAllInBound	*	*	*	Deny

These are **implicit**. If you add an explicit rule at priority 4000 to “deny all”, you shadow the implicit allows — exactly what broke our lab in iter 4 until we added explicit **AllowVnetInbound** (1000) and **AllowAzureLoadBalancerInbound** (1100) rules.

6.6.2 Visual: NSG rule evaluation

Incoming packet from 10.0.1.5 to 10.0.2.10:80

Check rule by priority (lowest first):

Priority 1000: AllowVnetInbound - MATCH → ALLOW
(stop checking)

Packet is allowed through

6.7 4.7 Load Balancers

A load balancer distributes incoming connections across multiple backend instances.

Public IP: 20.94.18.66

Load Balancer

Backend	Backend	Backend
Pod 1	Pod 2	Pod 3

Types: - **L4 Load Balancer** — operates on TCP/UDP, no understanding of HTTP - **L7 Load Balancer** — understands HTTP, can route by URL/headers (also called Application Gateway)

Azure Load Balancer = L4. Azure Application Gateway = L7. Istio ingress gateway = L7 (in our lab).

6.8 4.8 TLS/HTTPS

HTTPS = HTTP over TLS (Transport Layer Security). TLS provides: - **Encryption** — eavesdroppers can't read traffic - **Authentication** — verify you're talking to the real server - **Integrity** — detect tampering

The TLS handshake (simplified):

Client	Server
ClientHello (supported ciphers)	
ServerHello + Certificate	
(verify cert against trusted CA list)	
Generate session key, encrypt with server's public key	
Encrypted application data	

mTLS = mutual TLS. Client also presents a certificate. Used in service meshes for service-to-service auth. Istio does this by default.

6.9 4.9 Why this matters in interviews

“How does traffic flow from the internet to a Pod?”

Good answer: > “Internet → Azure Load Balancer (L4) at the public IP → istio-ingress Service (NodePort) → istio-ingress Pod (Envoy at L7) → matches a VirtualService by Host header → forwards to the destination Service's ClusterIP → CoreDNS resolves to a Pod IP → kube-proxy load-balances across Pod IPs of that Service → the request lands on the Pod, passes through its istio-proxy sidecar (mTLS termination), then hits the app container.”

6.10 4.10 Exercises

1. Use `ping`, `traceroute`, `dig`, `nslookup` against common domains
2. `nc -zv google.com 443` — test if a port is open
3. `curl -v https://example.com` — see the TLS handshake details
4. Calculate available IPs for /24, /27, /30

5. Read your laptop's `route -n` table

Chapter 7

Chapter 5: Containers and Docker

7.1 5.1 What problem do containers solve?

Before containers, deploying an app meant: - Install the OS - Install language runtime (Python, Node, etc.) - Install dependencies (libraries, system packages) - Copy your code - Configure environment variables - Start the process - Repeat on every server

This led to: - “Works on my machine” syndrome - Hours wasted on environment drift - Painful onboarding - Hard to reproduce production locally

A **container** packages the app + all its dependencies into a portable image. Run the image anywhere → identical behavior.

7.2 5.2 VMs vs Containers

VM model	Container model
App	App App App
Libs / runtime	Libs Libs Libs
Guest OS (full Linux)	Container runtime
Hypervisor	Host OS
Host OS	Hardware
Hardware	
VM size: GB	Container size: MB
VM boot: minutes	Container start: seconds
Isolation: hardware	Isolation: kernel namespaces

7.3 5.3 How containers work (the technical bits)

A container uses Linux kernel features: - **Namespaces** — isolate what processes can see (PID, network, mount, user, IPC, UTS) - **Cgroups** — limit CPU, memory, I/O - **Union filesystems** — layer filesystem changes efficiently (overlayFS)

A container is just a process on the host, isolated by these kernel features. There's no “container layer” running — it's all kernel-level.

7.4 5.4 Docker concepts

Dockerfile	docker build	Image	docker run	Container
			docker push	Registry (ACR, Docker Hub, ECR)
Term		Definition		
Image		A read-only template (layers of files + metadata)		
Container		A running instance of an image (writable layer on top)		
Dockerfile		Instructions to build an image		
Registry		Storage for images (Docker Hub, ACR, ECR)		
Tag		Label on an image, like <code>nginx:1.28-alpine</code>		
Layer		One step in an image build (each Dockerfile line is a layer, cached)		

7.5 5.5 Dockerfile anatomy

Our backend's Dockerfile (annotated):

```
# Stage 1: install dependencies in builder
FROM node:20-alpine3.22 AS deps          # base image (small Linux + Node 20)
WORKDIR /app                             # cwd for subsequent commands
COPY package.json package-lock.json ./   # copy from build context
RUN npm ci --omit=dev --no-audit          # install prod deps only

# Stage 2: minimal runtime image
FROM node:20-alpine3.22 AS runtime
WORKDIR /app
USER node                                # don't run as root
COPY --chown=node:node --from=deps /app/node_modules ./node_modules
COPY --chown=node:node server.js ./

ENV NODE_ENV=production
ENV PORT=5678
EXPOSE 5678                             # documentation, not actual port opening

CMD ["node", "server.js"]                # what runs when container starts
```

Why multi-stage? The final image doesn't include build tools, dev dependencies, or the build cache. Smaller image = faster pull, smaller attack surface.

7.6 5.6 Common Docker commands

Building

```
docker build -t myapp:1.0 .
docker build -t myapp:1.0 --target deps .    # build only the "deps" stage
```

Running

```
docker run -p 8080:80 nginx:alpine          # expose port 80 inside as 8080 on host
docker run -d -p 8080:80 nginx:alpine        # detached (background)
docker run -e MY_VAR=value nginx            # env var
docker run -v /host/path:/container/path nginx # volume mount
docker run --name webserver -d nginx        # name the container
```

Inspecting

```
docker ps                                # running containers
docker ps -a                             # all containers
docker logs webserver                    # see container output
docker exec -it webserver sh             # shell into running container
docker inspect webserver                 # full metadata
```

Cleanup

```
docker stop webserver
docker rm webserver                      # delete stopped container
docker rmi nginx:alpine                  # delete image
docker system prune                       # nuke unused stuff
```

7.7 5.7 Image layers and caching

Each Dockerfile instruction creates a layer. Docker caches layers. If line N didn't change, layers N are reused:

```
FROM node:20-alpine          # layer 1 (rarely changes)
COPY package.json ./         # layer 2 (changes when deps change)
RUN npm install               # layer 3 (changes only if layer 2 changed)
COPY . .                      # layer 4 (changes on every code edit)
```

Best practice: put rarely-changing stuff (deps) BEFORE frequently-changing stuff (code). Maximizes cache hits.

Our pipeline uses kaniko (Jenkins) and buildx (GitHub Actions) for builds. Both push intermediate layers to ACR as a cache.

7.8 5.8 Images and tags

Image reference structure:

[registry-host/]username/repo:tag[@digest]

Examples:

nginx	(defaults to: docker.io/library/nginx:latest)
nginx:1.28-alpine	(Docker Hub, tag 1.28-alpine)
gskplatacrn73d5y.azurecr.io/backend:latest	(ACR)
nginx@sha256:abcd...	(by digest, immutable)

Critical interview point: :latest tag is mutable — what :latest points to changes over time. Use immutable tags (git SHA, semver) for production. Use digest pins (@sha256:...) for ultimate guarantees.

Our lab moved from build-N (BUILD_NUMBER) to sha-abc1234 (git short SHA) for exactly this reason.

7.9 5.9 Container registries

Where images are stored. The big ones:

Registry	Use case
Docker Hub	Public images, free with rate limits
Azure Container Registry (ACR)	Azure ecosystem, integrates with AKS via managed identity
AWS ECR	AWS ecosystem
GitHub Container Registry (GHCR)	Free for open source, integrates with GitHub Actions
Quay.io	Red Hat ecosystem, supports CVE scanning
Self-hosted Harbor	On-prem

Authentication:

```
docker login mycr.azurecr.io
# Or via service principal:
docker login mycr.azurecr.io -u <sp-app-id> -p <sp-secret>
```

Our lab uses ACR. Kaniko/buildx use a Service Principal credential stored as a Kubernetes Secret.

7.10 5.10 Security best practices

Practice	Why
Use specific tags, not :latest	Reproducible builds
Run as non-root user (USER node)	Limit damage if container is compromised
Multi-stage builds	Smaller images, fewer CVEs
Scan images with Trivy/Snyk	Catch CVEs before deploy
Use distroless or alpine base	Fewer packages = fewer CVEs
Don't bake secrets into images	Use env vars or secret mounts
Sign images with Cosign	Verify origin at deploy time

Practice	Why
Pin base images by digest	Even tag pinning can shift
Drop Linux capabilities	Containers don't need most caps
Set resource limits	Prevent runaway containers

Our lab does: non-root, multi-stage, Trivy scan, alpine base. Missing: image signing, digest pinning (production gaps).

7.11 5.11 Why this matters in interviews

“Walk me through a Dockerfile review.”

Good answer: > “I’d look for: (1) specific base image tag, not latest. (2) Multi-stage build to keep final image small. (3) `USER non-root` for principle of least privilege. (4) Layer ordering — dependencies before code for cache efficiency. (5) `.dockerignore` to keep build context small. (6) `EXPOSE` and `CMD` clearly defined. Red flags: secrets in `ENV`, `chmod 777`, no health check, `apt-get install` without cleaning `/var/lib/apt/lists/`.”

7.12 5.12 Exercises

1. Install Docker locally (or use Docker Desktop)
2. Write a Dockerfile for a simple Python script
3. Build it, run it, expose a port, hit it with curl
4. Add multi-stage builds
5. Run `docker history myapp:1.0` and explain each layer
6. Scan it: `docker scout cves myapp:1.0`
7. Push to Docker Hub (free account)

Chapter 8

Chapter 6: Kubernetes Deep Dive

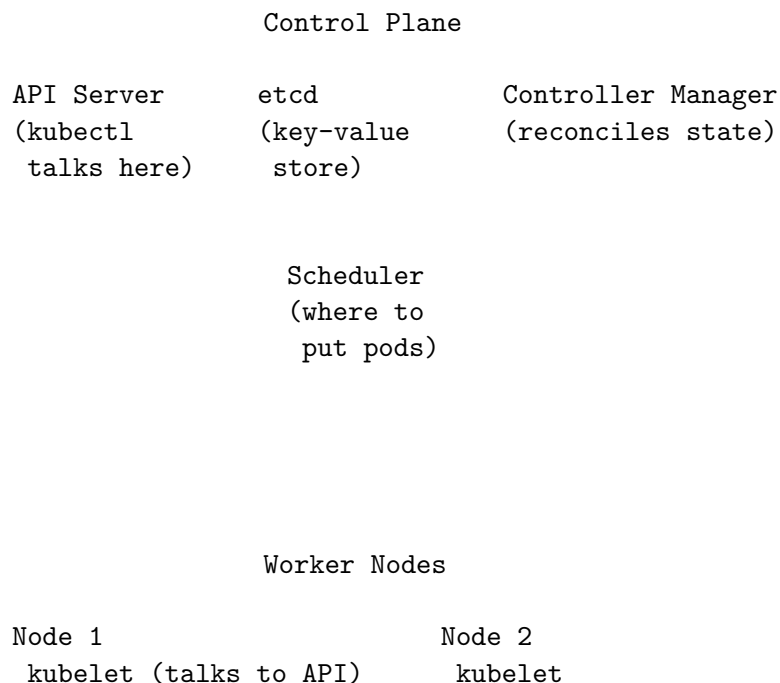
8.1 6.1 What Kubernetes is

Kubernetes (k8s) is a container orchestrator. You declare desired state (“I want 3 copies of this app running, expose port 80, restart if any die”), and Kubernetes makes it happen.

Why containers + orchestrator? - Manual `docker run` doesn’t scale beyond 1 server - Need: health checks, restarts, scheduling, load balancing, secrets management, networking between containers, rolling updates

Kubernetes solves all of these.

8.2 6.2 The architecture



kube-proxy (networking)	kube-proxy
container runtime	container runtime
(containerd / CRI-O)	
Pod	Pod

In AKS: the control plane is managed by Azure (free in Free SKU). You only see/pay for worker nodes.

8.3 6.3 Core objects

8.3.1 Pod

The smallest deployable unit. Contains 1+ containers that share network + storage.

```
apiVersion: v1
kind: Pod
metadata:
  name: my-pod
spec:
  containers:
    - name: nginx
      image: nginx:1.28-alpine
      ports:
        - containerPort: 80
```

You rarely create Pods directly. Use Deployments or StatefulSets which create Pods.

8.3.2 Deployment

Manages replicas of a Pod. Handles rolling updates.

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: frontend
spec:
  replicas: 3
  selector:
    matchLabels:
      app: frontend
  template:
    metadata:
      labels:
        app: frontend
    spec:
```

```

containers:
  - name: nginx
    image: myacr/frontend:sha-abc123

```

Lifecycle:

You apply Deployment

↓

Deployment creates ReplicaSet

↓

ReplicaSet creates 3 Pods

↓

You update image to sha-def456

↓

Deployment creates new ReplicaSet

↓

New ReplicaSet creates Pods one at a time

↓

Old Pods deleted one at a time

↓

After all Pods updated, old ReplicaSet kept (for rollback)

8.3.3 Service

A stable network endpoint for Pods. Pods come/go; Service stays.

```

apiVersion: v1
kind: Service
metadata:
  name: frontend
spec:
  type: ClusterIP                # internal only (default)
  selector:
    app: frontend                # matches Pod labels
  ports:
    - port: 80                  # service port
      targetPort: 80            # pod port

```

Service types: | Type | What | |---| | ClusterIP | Internal IP only (default) | | NodePort | Open same port on every node | | LoadBalancer | Cloud provider creates external LB | | ExternalName | DNS alias to external service |

8.3.4 ConfigMap and Secret

Inject config into Pods.

```

apiVersion: v1
kind: ConfigMap
metadata:
  name: app-config
data:

```

```

LOG_LEVEL: info
DB_HOST: postgres.dev.svc.cluster.local

apiVersion: v1
kind: Secret
metadata:
  name: db-creds
type: Opaque
data:
  password: c3VwZXJzZWNyZXQ=      # base64 encoded

```

Critical: Secrets are base64-encoded, NOT encrypted by default. Use external secret managers (Azure Key Vault, AWS Secrets Manager) for real secrets. Our lab still has hardcoded passwords in values.yaml — a known production gap.

8.3.5 Namespace

Logical isolation within a cluster.

```

apiVersion: v1
kind: Namespace
metadata:
  name: dev
  labels:
    istio-injection: enabled      # opt into Istio sidecar

```

Our lab: dev, prod, argocd, jenkins, istio-system, monitoring namespaces.

8.4 6.4 kubectl daily commands

```

# Context (which cluster)
kubectl config current-context
kubectl config use-context <name>

```

Reading

kubectl get pods	# in default namespace
kubectl get pods -n dev	# in dev namespace
kubectl get pods -A	# all namespaces
kubectl get pods -o wide	# extra columns
kubectl get pods -o yaml	# full YAML
kubectl describe pod my-pod	# events + details
kubectl logs my-pod	# logs
kubectl logs my-pod -c sidecar	# specific container
kubectl logs my-pod --previous	# previous instance's logs
kubectl logs -f my-pod	# follow

Writing

kubectl apply -f manifest.yaml	# create/update from YAML
kubectl delete -f manifest.yaml	
kubectl edit deployment frontend	# edit live (avoid in prod)

```

# Debugging
kubectl exec -it my-pod -- sh           # shell into container
kubectl port-forward svc/frontend 8080:80 # forward local port to service
kubectl run debug --rm -it --image=busybox -- sh # ephemeral debug pod

# Scaling
kubectl scale deployment frontend --replicas=5
kubectl rollout status deployment/frontend
kubectl rollout undo deployment/frontend # rollback

# Events
kubectl get events -A --sort-by='.lastTimestamp' | tail -20

```

8.5 6.5 Pod lifecycle

Pending
(scheduler decides where to place)

Running

Succeeded Failed

(gone)

Within Running, containers have: - **Init** containers — run before main containers - **Liveness probe** — “is the app alive?” → restart on failure - **Readiness probe** — “is the app ready for traffic?” → remove from Service load balancer until ready - **Startup probe** — “has it started yet?” → grace period before liveness kicks in

```

spec:
  containers:
  - name: backend
    readinessProbe:
      httpGet:
        path: /health
        port: 5678
      initialDelaySeconds: 2
      periodSeconds: 5

```

8.6 6.6 Pod scheduling

The scheduler picks a node based on: - **Resource requests** — does the node have enough CPU/memory free? - **Node selector** / **nodeAffinity** — does the node have required labels? - **Tolerations** / **taints** — can the pod tolerate the node’s taints? - **Pod affinity/anti-affinity** — should it be near or far from other pods?

```
spec:
  containers:
  - name: app
    resources:
      requests:
        cpu: 100m           # 0.1 CPU
        memory: 128Mi
      limits:
        cpu: 500m
        memory: 512Mi
```

Critical: `requests` is what the scheduler uses to decide placement. `limits` is the max the container can use. If you set requests too high, scheduling fails (“Insufficient cpu”). If limits too low, OOM kill.

Our lab gotcha: Default Istio sidecar requests were 100m CPU each. $8 \text{ pods} \times 100\text{m} = 800\text{m}$. On a 2 vCPU node with system pods taking 1100m, scheduler couldn’t fit them. Fix: trim sidecars to 10m via annotations.

8.7 6.7 Networking inside Kubernetes

Pod-to-Pod (same node)	via veth pair + Linux bridge
Pod-to-Pod (across nodes)	via overlay network (kubenet, Calico, Cilium)
Pod-to-Service	kube-proxy DNAT to a Pod IP
Pod-to-Internet	via node IP (SNAT)
External-to-Service	via LoadBalancer / Ingress

CNI options: - **kubenet** (Azure default) — simple, NAT-based, no direct pod-to-pod across cluster boundaries - **Azure CNI** — pods get VNet IPs, can route to other Azure resources - **Calico**, **Cilium** — advanced, network policies, eBPF

Our lab uses kubenet (Free Trial constraint).

8.8 6.8 StatefulSet vs Deployment

Deployment:	StatefulSet:
Pods are cattle	Pods are pets
pod-7c8d-xyz1	mydb-0
pod-7c8d-xyz2	mydb-1
pod-7c8d-xyz3	mydb-2
	↑ ordinal, stable
Random order start	Ordered start (0 → 1 → 2)
Random delete	Reverse delete (2 → 1 → 0)
Shared PVC	Per-pod PVC

Use StatefulSet for databases (each pod has identity + persistent storage). Use Deployment for stateless apps.

Our lab: Postgres is a StatefulSet, frontend/backend are Rollouts (similar to Deployment).

8.9 6.9 PersistentVolume and PersistentVolumeClaim

StorageClass "managed-csi" (Azure disk)

(provisions on PVC creation)

PersistentVolume 10Gi disk in Azure

(bound)

PersistentVolumeClaim "postgres-data"

(mounted)

Pod /var/lib/postgresql/data

```
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: postgres-data
spec:
  accessModes: [ReadWriteOnce]
  storageClassName: managed-csi
  resources:
    requests:
      storage: 1Gi
```

Access modes: - ReadWriteOnce — one node at a time (Azure disk) - ReadOnlyMany — many nodes can read - ReadWriteMany — many nodes can write (Azure Files, NFS)

Our lab: managed-csi for Postgres (RWO), azurefile-csi for npm/trivy cache (RWX).

8.10 6.10 RBAC (Role-Based Access Control)

ServiceAccount ← pod identity

bound to

Role / ClusterRole ← what can be done

RoleBinding / ClusterRoleBinding ← grants the role to the ServiceAccount

Example: Jenkins SA needs to do helm upgrade:

```
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRole
metadata:
  name: jenkins-deployer
rules:
  - apiGroups: [""]
    resources: ["pods", "services", "secrets"]
    verbs: ["get", "list", "watch", "create", "update", "patch", "delete"]
```

```

apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata:
  name: jenkins-deployer-dev
  namespace: dev
subjects:
- kind: ServiceAccount
  name: jenkins
  namespace: jenkins
roleRef:
  kind: ClusterRole
  name: jenkins-deployer

```

Rule of thumb: ClusterRole is reusable, RoleBinding namespaces it.

8.11 6.11 Common pod failure modes

Pending	Insufficient resources, node selector mismatch, PVC not bound
Init:0/1	Init container failing
CrashLoopBackOff	Container exits, k8s restarts, exits, ...
ImagePullBackOff	Can't pull the image (wrong tag, no creds, network)
OOMKilled	Used more memory than the limit
Error	Container exit code != 0
Completed	Job done (for one-shot pods)

How to debug:

```

kubectl describe pod <pod>           # events at bottom
kubectl logs <pod>                   # what app printed
kubectl logs <pod> --previous         # from last crash
kubectl get events -n <ns> --sort-by='.lastTimestamp'

```

8.12 6.12 Why this matters in interviews

“How would you debug a pod stuck in Pending?”

Good answer: > “First `kubectl describe pod <name>` — events at the bottom usually tell you. Common: ‘Insufficient cpu’ (requests too high or cluster full), ‘FailedScheduling: 0/2 nodes match’ (node selector or taints), ‘pvc not bound’ (StorageClass missing or PVC mistyped). I’d check resource requests vs cluster capacity, look at the node’s allocatable resources, verify any node affinity rules, and ensure the namespace has any required SCCs/PSPs.”

8.13 6.13 Exercises

1. Install `kubectl`, configure for a local cluster (kind, minikube, k3d)
2. Apply a Deployment + Service YAML, expose it, hit it with `curl`
3. Create a ConfigMap and mount it in a Pod
4. Trigger an OOM by setting limits low and stressing memory
5. Use `kubectl port-forward` to access an internal service

6. Write an RBAC policy that lets a ServiceAccount only read pods
-

Chapter 9

Chapter 7: Infrastructure as Code with Terraform

9.1 7.1 What is IaC?

Instead of clicking through cloud consoles, **describe infrastructure as code**. Benefits:

- **Reproducible** — same code produces same infrastructure every time
- **Reviewable** — diff in a PR, get approval
- **Versioned** — Git history shows who changed what
- **Disaster recovery** — destroy and re-create from scratch
- **Self-documenting** — code IS the documentation

9.2 7.2 Terraform basics

Terraform uses HCL (HashiCorp Configuration Language):

```
# Provider - which cloud
terraform {
  required_providers {
    azurerm = {
      source  = "hashicorp/azurerm"
      version = "~> 3.0"
    }
  }
}

provider "azurerm" {
  features {}
}

# Resource - something to create
resource "azurerm_resource_group" "main" {
  name = "my-rg"
```

```

location = "eastus2"
tags = {
  Environment = "lab"
}
}

# Output - values to surface after apply
output "rg_id" {
  value = azurerm_resource_group.main.id
}

```

9.3 7.3 Terraform workflow

```

terraform  → terraform  → terraform
init       plan         apply

```

Download provider	Show diff	Make changes
Set up backend	(what would change)	(creates/updates cloud resources)

In a directory with .tf files

```

terraform init           # download provider, set up backend
terraform plan           # preview changes (NO changes made)
terraform plan -out=tfplan # save plan to a file
terraform apply tfplan   # apply the saved plan (safer in CI)
terraform apply          # plan + apply with prompt
terraform destroy        # delete everything

```

State management

```

terraform state list      # what's tracked
terraform state show <res> # details of a resource
terraform state rm <res>  # untrack (resource stays in cloud)
terraform import <res> <id> # adopt existing cloud resource into state

```

Other

```

terraform fmt           # format code
terraform validate      # syntax check
terraform output         # show output values

```

9.4 7.4 State

Terraform stores the current state of your infrastructure in `terraform.tfstate`. This is critical:

- Maps resources in HCL to real-world IDs
- Required to compute diffs

- Contains sensitive data (passwords, keys)

9.4.1 Local vs Remote state

Local state: terraform.tfstate on your laptop. Bad for teams.

Remote state: stored in Azure Storage, S3, etc. Enables: - Team collaboration (locking prevents simultaneous applies) - Backup - CI/CD pipelines can apply

Our lab uses Azure Storage:

```
terraform {
  backend "azurerm" {
    resource_group_name = "gskplat-rg-tfstate"
    storage_account_name = "gskplattfstatey2gvg3"
    container_name      = "tfstate"
    key                  = "live.terraform.tfstate"
  }
}
```

9.4.2 The chicken-and-egg problem

You need a state SA to use remote state, but you need Terraform to create the SA. Solution: **bootstrap** with local backend.

```
terraform/
  bootstrap/      # local backend - creates the state SA
  main.tf
  live/           # remote backend - uses the SA created by bootstrap
  main.tf
```

Run bootstrap once with local state, then migrate live to remote.

9.5 7.5 Variables and outputs

```
# variables.tf
variable "location" {
  type      = string
  default   = "eastus2"
  description = "Azure region"
}

variable "tags" {
  type      = map(string)
  default = {
    Project = "azure-platform-lab"
  }
}

# Use it
```

```
resource "azurerm_resource_group" "main" {
  name      = "rg-${var.location}"
  location  = var.location
  tags      = var.tags
}
```

Set values via terraform.tfvars:

```
location = "westeurope"
tags = {
  Project = "demo"
  Owner   = "alice"
}
```

Or env vars: `TF_VAR_location=westeurope terraform apply`

9.6 7.6 Modules

Modules are reusable bundles of .tf files.

```
terraform/
  modules/
    network/
      main.tf
      variables.tf
      outputs.tf
    aks/
  live/
    main.tf      # consumes modules

# live/main.tf
module "network" {
  source = "../modules/network"

  resource_group_name = azurerm_resource_group.main.name
  location             = "eastus2"
  vnet_cidr            = "10.0.0.0/16"
}

module "aks" {
  source = "../modules/aks"

  resource_group_name = azurerm_resource_group.main.name
  vnet_subnet_id      = module.network.aks_subnet_id
}
```

Our lab uses 3 modules: `network`, `acr`, `aks`. Each gets composed in `live/`.

9.7 7.7 State locking

When you run `terraform apply`, the backend locks the state. Concurrent applies are blocked.

```
# Person A
terraform apply          # acquires lock

# Person B
terraform apply          # ERROR: state is locked
```

Azure Storage uses blob leases. AWS S3 uses DynamoDB. Locks auto-release on apply complete, even on crash (with timeout).

9.8 7.8 Data sources

Read existing cloud resources without managing them:

```
data "azurerm_subscription" "current" {}

resource "azurerm_role_assignment" "example" {
  scope      = data.azurerm_subscription.current.id
  role       = "Reader"
  principal_id = "...
}
```

9.9 7.9 Common gotchas

Gotcha	Fix
Apply takes forever	Most cloud resources are async. AKS takes ~10 min, ACR takes ~30s
Provider permissions	SP needs both Contributor AND User Access Administrator for role assignments
Drift from manual changes	<code>terraform plan</code> shows them. Either accept or refactor HCL
State corruption	Backup <code>terraform.tfstate</code> before risky operations
Sensitive in state	Use <code>sensitive = true</code> , never commit state to git
Implicit dependencies	Terraform infers from references. Use <code>depends_on</code> for explicit
Resource recreation	Some changes force replace (e.g., changing VNet CIDR). Plan shows <code>-/+</code>

Our lab's RBAC gotcha was iter 2: SP had **Contributor** only, the AKS module tried to create role assignments, got `AuthorizationFailed: Microsoft.Authorization/roleAssignments/write denied`. Added **User Access Administrator** and it worked.

9.10 7.10 Production patterns

Pattern 1 - Workspace per environment

```
terraform workspace new dev
terraform workspace new prod
Use ${terraform.workspace} in HCL
```

Pattern 2 - Directory per environment (recommended)

```
terraform/
  live/
    dev/
      main.tf
      terraform.tfvars
    prod/
      main.tf
      terraform.tfvars
```

Pattern 3 - One state per service

```
terraform/
  networking/
  identity/
  kubernetes/
  data/
Each has its own state file
```

Our lab uses pattern 1.5 — single `live/` with tfvars per env (not fully separated). Production gap.

9.11 7.11 Why this matters in interviews

“Why two Terraform directories (bootstrap + live)?”

Good answer: > “Chicken-and-egg. Terraform needs a backend before using one. Bootstrap creates the state Storage Account with a local backend (its state stays on the developer’s laptop or a one-time CI run). Live then references that SA as its remote backend. Once bootstrap is done, you essentially never touch it again. Live is where day-to-day infrastructure changes happen.”

9.12 7.12 Exercises

1. Install Terraform: `brew install terraform` or download from terraform.io
 2. Write HCL for an Azure Resource Group, apply it
 3. Add a Storage Account inside that RG
 4. Migrate from local to remote backend
 5. Refactor to use a module
 6. Try `terraform import` on an existing cloud resource
 7. Inspect the state: `terraform state pull | jq`
-

Chapter 10

Chapter 8: Helm — Package Manager for Kubernetes

10.1 8.1 The problem Helm solves

A typical app needs many k8s objects: Deployment + Service + ConfigMap + Secret + Ingress + PVC. Writing them all is tedious. Templating differences per environment is hard.

Helm is a package manager for Kubernetes. A **chart** is a package containing templates + default values.

10.2 8.2 Chart structure

```
mychart/
  Chart.yaml           # chart metadata
  values.yaml          # default values
  templates/
    deployment.yaml
    service.yaml
    configmap.yaml
    _helpers.tpl       # named templates
  charts/              # sub-charts (dependencies)
  README.md
```

10.3 8.3 Templates with values

Template file templates/deployment.yaml:

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: {{ .Release.Name }}-{{ .Chart.Name }}
spec:
  replicas: {{ .Values.replicaCount }}
```



```

selector:
  matchLabels:
    app: {{ .Chart.Name }}
template:
  metadata:
    labels:
      app: {{ .Chart.Name }}
  spec:
    containers:
      - name: app
        image: "{{ .Values.image.repository }}:{{ .Values.image.tag }}"
        resources:
          {{- toYaml .Values.resources | nindent 12 }}

```

Values file values.yaml:

```

replicaCount: 3
image:
  repository: nginx
  tag: 1.28-alpine
resources:
  requests:
    cpu: 100m
    memory: 128Mi

```

Helm interpolates {{ }} blocks. Sprig functions for string/list manipulation.

10.4 8.4 Common Helm commands

Add a repo

```

helm repo add bitnami https://charts.bitnami.com/bitnami
helm repo update

```

Search for charts

```

helm search repo bitnami/nginx
helm search hub nginx           # search Artifact Hub

```

Install

```

helm install my-app bitnami/nginx
helm install my-app bitnami/nginx --values myvalues.yaml
helm install my-app bitnami/nginx --set replicaCount=5
helm install my-app . --namespace dev --create-namespace

```

Inspect

```

helm list                                # releases
helm list -A                             # all namespaces
helm get values my-app                   # the values used
helm get manifest my-app                 # rendered YAML
helm history my-app                       # revision history

```

```

helm template ./mychart          # render without installing (debug)

# Update
helm upgrade my-app bitnami/nginx --values myvalues.yaml
helm upgrade --install my-app ./mychart    # install if missing, upgrade if exists

# Rollback
helm rollback my-app 1           # to revision 1

# Uninstall
helm uninstall my-app

```

10.5 8.5 Subcharts / umbrella charts

Our lab uses an umbrella chart:

```

kubernetes/apps/three-tier/
  Chart.yaml          # umbrella - depends on subcharts
  values.yaml         # defaults for umbrella
  values-dev.yaml
  values-prod.yaml
  charts/
    frontend/        # subchart
    backend/         # subchart
    database/        # subchart

```

Chart.yaml:

```

apiVersion: v2
name: three-tier
version: 0.1.0
dependencies:
  - name: frontend
    version: 0.1.0
    repository: file:///./charts/frontend
  - name: backend
    version: 0.1.0
  - name: database
    version: 0.1.0

```

To override a subchart value, nest under the subchart name:

```

# values-dev.yaml
frontend:          # ← passes to frontend subchart
  image:
    tag: sha-abc123
backend:
  image:
    tag: sha-abc123

```

10.6 8.6 Conditionals and loops

```
{{- if .Values.blueGreen.enabled }}
apiVersion: argoproj.io/v1alpha1
kind: Rollout
# ...
{{- else }}
apiVersion: apps/v1
kind: Deployment
# ...
{{- end }}

{{- range .Values.containers }}
- name: {{ .name }}
  image: {{ .image }}
{{- end }}

{{- range $key, $val := .Values.env }}
- name: {{ $key }}
  value: {{ $val | quote }}
{{- end }}
```

Our lab uses this to toggle between Deployment and Rollout based on `blueGreen.enabled`.

10.7 8.7 Helm hooks

Run jobs at specific points in the lifecycle:

```
apiVersion: batch/v1
kind: Job
metadata:
  name: db-migrate
  annotations:
    helm.sh/hook: pre-install,pre-upgrade
    helm.sh/hook-weight: "-5"
    helm.sh/hook-delete-policy: hook-succeeded
```

Common hooks: - `pre-install` — before install - `post-install` — after install - `pre-upgrade` / `post-upgrade` - `pre-delete` / `post-delete`

10.8 8.8 Helm vs Kustomize

	Helm	Kustomize
Mechanism	Templates + values	Patches on base manifests
Language	Go templates (powerful, hard)	None (pure YAML)
Versioning	Charts published to registry	None natively
Pkg mgmt	Yes (helm install)	No
Learning curve	Steeper	Gentler

Industry Very widely used Built into `kubectl`

Most teams use Helm for installs (Nginx Ingress, cert-manager, ArgoCD, Prometheus). Some use Kustomize for in-house apps.

10.9 8.9 Why this matters in interviews

“How do you manage configuration across environments?”

Good answer: > “Helm chart with environment-specific values files. We have `values-dev.yaml` and `values-prod.yaml` overriding the defaults. The chart itself is generic — toggle features via `--set` or values files. Image tags get injected by CI (writing to `values.yaml` + committing to Git). For sensitive values, External Secrets Operator pulls from Azure Key Vault into k8s Secrets the chart references.”

10.10 8.10 Exercises

1. `helm install my-nginx bitnami/nginx`
 2. Override default replicas with `--set`
 3. Write a chart from scratch for a simple app
 4. Add a conditional template
 5. Use `helm template . --debug` to inspect rendering
 6. Create an umbrella chart with two subcharts
 7. Try `helm rollback` after an upgrade
-

Chapter 11

Chapter 9: Azure Specifics for Platform Engineers

11.1 9.1 Azure resource hierarchy

Tenant (Microsoft Entra ID)

Management Groups (optional, multi-sub orgs)

Subscription (billing boundary)

Resource Group (logical container)

Resource (VM, AKS, ACR, etc.)

A **subscription** is where you pay. A **resource group** is where you organize. Everything lives in a resource group.

11.2 9.2 Authentication models

11.2.1 Azure CLI login (interactive)

```
az login                                # opens browser
az account show
az account list
az account set --subscription <id>
```

11.2.2 Service Principal (for automation)

A non-human identity. Has: - Application (Client) ID - Tenant ID - Client Secret OR certificate

```
# Create SP with Contributor on a subscription
az ad sp create-for-rbac \
  --name sp-myapp-terraform \
  --role Contributor \
```

```
--scopes /subscriptions/<sub-id>
```

```
# Output:
# {
#   "appId": "...",
#   "displayName": "...",
#   "password": "...",      + client secret
#   "tenant": "..."
# }
```

Use the SP from Terraform:

```
export ARM_CLIENT_ID=<appId>
export ARM_CLIENT_SECRET=<password>
export ARM_TENANT_ID=<tenant>
export ARM_SUBSCRIPTION_ID=<sub-id>
```

11.2.3 Workload Identity (modern, no secrets)

Modern apps in AKS use Workload Identity: - AKS has OIDC issuer - Federated credential ties a k8s ServiceAccount to an Azure AD Application - Pod gets a JWT token, exchanges for Azure AD access token - No long-lived secrets

Our lab still uses SP credentials (a known production gap).

11.3 9.3 RBAC roles

Role	Permissions
Owner	Full access + can grant access
Contributor	Full access except role assignments
User Access Administrator	Manage role assignments
Reader	Read-only
Built-in service roles	e.g., AKS Admin, AcrPull, AcrPush

The gotcha our lab hit: To create role assignments in Terraform, the SP needs **both** Contributor AND User Access Administrator. Contributor alone fails with `Microsoft.Authorization/roleAssignments/write` denied.

11.4 9.4 Key Azure services for platform engineers

11.4.1 AKS (Azure Kubernetes Service)

Managed Kubernetes. Control plane is free in Free SKU.

Key concepts: - **Node pool** — group of VMs running the same SKU - **System node pool** (required) — runs core services - **User node pool** (optional) — for your workloads - **Cluster autoscaler** — adds/removes nodes - **OIDC issuer** — required for Workload Identity - **Public vs Private** — API endpoint exposure

```

# Get credentials
az aks get-credentials -g <rg> -n <cluster>

# Scale
az aks scale -g <rg> -n <cluster> --node-count 3 --nodepool-name default

# Stop (save $$, control plane stays free)
az aks stop -g <rg> -n <cluster>
az aks start -g <rg> -n <cluster>

```

11.4.2 ACR (Azure Container Registry)

Docker image registry.

```

# Login (uses your az login)
az acr login -n <acr-name>

# Push
docker tag myapp:1.0 myacr.azurecr.io/myapp:1.0
docker push myacr.azurecr.io/myapp:1.0

# List images
az acr repository list -n <acr-name>
az acr repository show-tags -n <acr-name> --repository myapp

```

ACR attach to AKS (the right way):

```
az aks update -g <rg> -n <cluster> --attach-acr <acr-name>
```

This assigns the AKS kubelet's managed identity the AcrPull role. No image pull secrets needed.

11.4.3 Virtual Network (VNet)

Private network within Azure.

```

resource "azurerm_virtual_network" "main" {
  name                = "myvnet"
  resource_group_name = "myrg"
  location            = "eastus2"
  address_space       = ["10.0.0.0/16"]
}

resource "azurerm_subnet" "aks" {
  name                = "snet-aks"
  resource_group_name = "myrg"
  virtual_network_name = azurerm_virtual_network.main.name
  address_prefixes     = ["10.0.1.0/24"]
}

```

11.4.4 Log Analytics Workspace

Centralized logging. AKS sends logs/metrics via the OMS agent (Container Insights).

11.4.5 Storage Account

Multiple services in one: - **Blob storage** — like S3 - **File storage** — SMB shares - **Queue storage** — message queue - **Table storage** — key-value store

Used by Terraform for state, by Velero for backups, by Azure Files PVCs.

11.5 9.5 Free Trial constraints

Constraint	Impact
\$200 credit, 30 days	Time pressure
4 vCPU per family per region quota	Can't run 2 separate clusters
Some SKUs blocked (B-series in some regions)	Forced to D2s_v3 (\$0.10/hr instead of \$0.04/hr)
Free SKU AKS limits	Single uptime SLA, no SLA

Our lab compensates with: - Single cluster + namespaces (instead of multi-cluster) - `az aks stop` overnight (saves ~75% cost) - Sidecar resource trimming

11.6 9.6 Common az CLI commands

Account

```
az account show
az account list-locations -o table
```

Resource Groups

```
az group create -n myrg -l eastus2
az group list -o table
az group delete -n myrg --yes
```

AKS

```
az aks create -g rg -n cluster --node-count 1 --node-vm-size Standard_D2s_v3 ...
az aks list -o table
az aks get-credentials -g rg -n cluster
```

ACR

```
az acr create -g rg -n myacr --sku Basic
az acr login -n myacr
az acr show -n myacr
```

Identity / role

```
az ad sp create-for-rbac --name myapp
az role assignment create --assignee <appid> --role Reader --scope /subscriptions/<sub>
```


Cost

```
az consumption usage list --output table
```

11.7 9.7 AKS networking modes

Mode	What
kubenet (default)	Pods get a non-routable IP, NATed via node IP for outbound. Simple. Limitation: pods can't directly route to other Azure resources without NAT
Azure CNI	Pods get VNet IPs. Routable to other Azure resources. Costs more IPs from subnet
Azure CNI Overlay	New default. Pods get overlay IPs, node IPs in VNet. Best of both
BYOCNI	Cilium, Calico — bring your own

Our lab uses kubenet (simpler, Free Trial-friendly). Production would use Azure CNI Overlay.

11.8 9.8 Why this matters in interviews

“When would you choose AKS over EKS or GKE?”

Good answer: > “Mostly business reasons — alignment with existing Azure spend, Entra ID integration for SSO, ExpressRoute connections to on-prem, Microsoft enterprise agreements. Technically all three are mature. AKS has free control plane in non-paid tier, native Workload Identity, and tight Defender for Containers integration. EKS has more mature options (Fargate, multi-cluster Karpenter ecosystem). GKE invented Kubernetes and has best-in-class autopilot mode. I'd pick based on where the rest of the stack already lives.”

11.9 9.9 Exercises

1. Create a free Azure account, install **az** CLI, **az login**
2. Create a Service Principal, set **ARM_*** env vars
3. Create a small AKS cluster manually with **az aks create**
4. Connect with **az aks get-credentials**
5. Push a Docker image to ACR
6. **az aks stop** and verify cost stops accumulating
7. Read costs with **az consumption usage list**

Chapter 12

Part II — Building the Lab

Chapter 13

Chapter 10: Lab Architecture Overview

13.1 10.1 What we're building

A production-pattern AKS lab:

Azure Subscription

gskplat-rg-platform (Resource Group)

AKS cluster (2× D2s_v3)

Namespaces:

- istio-system (mesh + ingress gateway)
- argocd (GitOps controller)
- argo-rollouts (progressive delivery controller)
- jenkins (CI option A)
- monitoring (Prometheus + Grafana)
- dev (3-tier app)
- prod (3-tier app)

ACR	VNet (10.0.0.0/16)
container	• snet-aks (10.0.1.0/24)
registry	• snet-apps (10.0.2.0/24)
	• snet-mgmt (10.0.3.0/27)

Log Analytics WS ← OMS agent on every node

gskplat-rg-tfstate (Resource Group)

Storage Account ← Terraform remote state
(tfstate container)

Terraform (IaC)

helm + kubectl

Developer VM

CI/CD systems

- Terraform CLI
- Azure CLI
- kubectl
- helm
- git

- Jenkins (in-cluster)
- GitHub Actions (SaaS)
- CircleCI (SaaS)

13.2 10.2 The 6 iterations

Each iteration ships a working slice:

Iter 1	Bootstrap state + Network	→ RG, LAW, VNet, NSGs
Iter 2	ACR + AKS	→ registry + cluster
Iter 3	Platform components	→ Istio, ArgoCD, Jenkins
Iter 4	Three-tier app (placeholders)	→ Helm chart, namespaces, VirtualServices
Iter 5	Real app code + Jenkins CI	→ kaniko builds, helm deploys
Iter 6	GitHub Actions + Argo Rollouts + Prometheus	→ GitOps, blue/green, analysis gates
Iter 7	CircleCI (third CI for portfolio)	→ same GitOps pattern, different toolcha

13.3 10.3 The lab tells three CI stories

Story	Tool	Pattern
Old school	Jenkins	In-cluster, helm directly
Modern SaaS	GitHub Actions	GitOps via commit-to-Git
Multi-team SaaS	CircleCI	GitOps with Contexts (RBAC for secrets)

You can speak to any of them in an interview.

13.4 10.4 Why this architecture

Choices and rationales:

Choice	Why
Single AKS cluster, namespace isolation	Free Trial quota constraint
kubenet not Azure CNI	Simpler IPs, Free Trial-friendly
Istio not Linkerd	More widely deployed in interviews
ArgoCD not Flux	More widely deployed; better UI
Argo Rollouts not Flagger	Pairs cleanly with ArgoCD (same org)
Helm not Kustomize	More widely deployed
Two-gate analysis (min-traffic + success)	Prevents false positives from zero-traffic
Git SHA as image tag	Immutable, traceable, enables build-once-deploy-many

Chapter 14

Chapter 11: Iteration 1 — Bootstrap and Network

14.1 11.1 The chicken-and-egg fix

You need a state SA before you can use one. Bootstrap solves this:

```
terraform/
  bootstrap/          ← local state, runs ONCE
    main.tf           (creates the state SA)
    variables.tf
    outputs.tf
    terraform.tfvars.example
  live/              ← remote state, your daily work
    main.tf           (uses the SA from bootstrap)
    ...
```

14.1.1 Bootstrap workflow

```
1. cd terraform/bootstrap
2. cp terraform.tfvars.example terraform.tfvars (edit owner email)
3. terraform init                               (local backend)
4. terraform apply                             (creates SA in cloud)
5. Note the `backend_init_command_live` output
6. cd ../live
7. terraform init -backend-config="storage_account_name=..."
   -backend-config="container_name=tfstate"
   ...
   (now using remote state)
```

After this, you should NEVER need to touch bootstrap again. It just sits there.

14.2 11.2 The bootstrap/main.tf

```
terraform {
  required_version = ">= 1.5"
  required_providers {
    azurerm = { source = "hashicorp/azurerm", version = "~> 3.0" }
    random  = { source = "hashicorp/random",   version = "~> 3.5" }
  }
}

provider "azurerm" {
  features {}
}

# Random suffix so the SA name is globally unique
resource "random_string" "sa_suffix" {
  length  = 6
  upper   = false
  special = false
  numeric = true
}

resource "azurerm_resource_group" "tfstate" {
  name      = "${var.project}-rg-tfstate"
  location  = var.location
  tags      = var.tags
}

resource "azurerm_storage_account" "tfstate" {
  name                                = "${var.project}tfstate${random_string.sa_suffix.result}"
  resource_group_name                 = azurerm_resource_group.tfstate.name
  location                            = azurerm_resource_group.tfstate.location
  account_tier                        = "Standard"
  account_replication_type            = "LRS"
  allow_nested_items_to_be_public     = false
  min_tls_version                     = "TLS1_2"
  tags                                = var.tags
}

resource "azurerm_storage_container" "tfstate" {
  name                = "tfstate"
  storage_account_id  = azurerm_storage_account.tfstate.id
  container_access_type = "private"
}
```

Key choices explained: - **LRS** (Locally Redundant Storage) — cheapest, 3 copies within a single data center. State files are small and recoverable - **Random suffix** — Storage Account names must be globally unique - **min_tls_version = "TLS1_2"** — security best practice

14.3 11.3 The network module

```
# modules/network/main.tf

resource "azurerm_virtual_network" "main" {
  name                = "${var.project}-vnet-shared"
  resource_group_name = var.resource_group_name
  location            = var.location
  address_space       = [var.vnet_cidr]
  tags               = var.tags
}

# Subnet for AKS
resource "azurerm_subnet" "aks" {
  name                = "snet-aks"
  resource_group_name = var.resource_group_name
  virtual_network_name = azurerm_virtual_network.main.name
  address_prefixes     = [cidrsubnet(var.vnet_cidr, 8, 1)] # /24 from /16
}

# Subnet for management
resource "azurerm_subnet" "mgmt" {
  name                = "snet-mgmt"
  resource_group_name = var.resource_group_name
  virtual_network_name = azurerm_virtual_network.main.name
  address_prefixes     = [cidrsubnet(var.vnet_cidr, 11, 24)] # /27 from /16
}
```

14.4 11.4 NSGs — the explicit deny gotcha

We added NSGs with explicit deny-all rules. This broke pod-to-pod traffic across nodes.

```
resource "azurerm_network_security_group" "aks" {
  name                = "nsg-aks"
  resource_group_name = var.resource_group_name
  location            = var.location

  # CRITICAL: explicit allows BEFORE explicit deny
  security_rule {
    name                = "AllowVnetInbound"
    priority            = 1000
    direction           = "Inbound"
    access              = "Allow"
    protocol            = "*"
    source_port_range    = "*"
    destination_port_range = "*"
    source_address_prefix = "VirtualNetwork"
    destination_address_prefix = "VirtualNetwork"
  }
}
```



```

}

security_rule {
    name                = "AllowAzureLoadBalancerInbound"
    priority             = 1100
    direction           = "Inbound"
    access              = "Allow"
    protocol            = "*"
    source_port_range   = "*"
    destination_port_range = "*"
    source_address_prefix = "AzureLoadBalancer"
    destination_address_prefix = "*"
}

# Explicit deny at high priority
security_rule {
    name                = "DenyAllInboundExplicit"
    priority             = 4000
    direction           = "Inbound"
    access              = "Deny"
    protocol            = "*"
    source_port_range   = "*"
    destination_port_range = "*"
    source_address_prefix = "*"
    destination_address_prefix = "*"
}
}

```

14.4.1 Why this matters

Azure's implicit allows are at priority 65000+ (lowest priority). An explicit Deny at priority 4000 fires FIRST. Without the explicit allows at 1000/1100, you've effectively denied all cluster-internal traffic.

This is a real interview question: "Why would adding a Deny rule break a working cluster?"

14.5 11.5 Outputs

```

output "vnet_id" {
    value = azurerm_virtual_network.main.id
}

output "aks_subnet_id" {
    value = azurerm_subnet.aks.id
}

output "law_id" {
    value = azurerm_log_analytics_workspace.shared.id
}

```

```
}
```

Other modules consume these outputs by referring to `module.network.aks_subnet_id`.

14.6 11.6 Apply iter 1

```
cd terraform/live
cp terraform.tfvars.example terraform.tfvars
# Edit: set admin_ip_cidr to your /32
terraform init      # paste backend command from bootstrap output
terraform plan      # expect ~12 resources to add
terraform apply
```

After iter 1: RG, LAW, VNet, 3 subnets, 3 NSGs.

Cost so far: < \$1/month.

Chapter 15

Chapter 12: Iteration 2 — ACR and AKS

15.1 12.1 ACR module

```
# modules/acr/main.tf

resource "random_string" "acr_suffix" {
  length  = 6
  upper   = false
  special = false
  numeric = true
}

resource "azurerm_container_registry" "main" {
  name                        = "${var.project}acr${random_string.acr_suffix.result}"
  resource_group_name        = var.resource_group_name
  location                   = var.location
  sku                        = "Basic"           # Cheapest tier
  admin_enabled              = false             # Force RBAC auth (security best practice)
  tags                       = var.tags
}
```

`admin_enabled = false` is important — it forces all auth through Azure AD, not via a shared username/password.

15.2 12.2 AKS module

```
# modules/aks/main.tf

resource "azurerm_kubernetes_cluster" "main" {
  name                        = "${var.project}-aks-shared"
  resource_group_name        = var.resource_group_name
  location                   = var.location
}
```

```

dns_prefix          = "${var.project}-aks"
kubernetes_version = "1.34"

default_node_pool {
  name          = "system"
  node_count    = var.node_count
  vm_size       = "Standard_D2s_v3"
  vnet_subnet_id = var.aks_subnet_id
  type          = "VirtualMachineScaleSets"
  only_critical_addons_enabled = false # allow user workloads (single pool)
}

identity {
  type = "SystemAssigned"
}

network_profile {
  network_plugin = "kubenet"          # Free Trial-friendly
  load_balancer_sku = "standard"
  service_cidr    = "172.16.0.0/16"
  dns_service_ip = "172.16.0.10"
}

# API server access - restrict to your IP only
api_server_access_profile {
  authorized_ip_ranges = [var.admin_ip_cidr]
}

# OIDC issuer - required for Workload Identity (even if you don't use it now)
oidc_issuer_enabled = true
workload_identity_enabled = true

# Log Analytics integration
oms_agent {
  log_analytics_workspace_id = var.law_id
}

sku_tier = "Free"      # Free SKU, single uptime SLA
tags      = var.tags
}

# Grant AKS kubelet identity permission to pull from ACR
resource "azurerm_role_assignment" "aks_to_acr" {
  scope          = var.acr_id
  role_definition_name = "AcrPull"
  principal_id    = azurerm_kubernetes_cluster.main.kubelet_identity[0].object_id
}

```

```
# Grant Terraform SP cluster admin for kubectl access
resource "azurerm_role_assignment" "sp_to_aks" {
  scope           = azurerm_kubernetes_cluster.main.id
  role_definition_name = "Azure Kubernetes Service Cluster Admin Role"
  principal_id     = var.terraform_sp_object_id
}
```

15.2.1 The RBAC gotcha

Without User Access Administrator on the SP, azurerm_role_assignment fails with:

Microsoft.Authorization/roleAssignments/write is not allowed

Add UAA to the SP at subscription scope:

```
az role assignment create \
  --assignee <sp-app-id> \
  --role "User Access Administrator" \
  --scope /subscriptions/<sub-id>
```

15.3 12.3 Apply iter 2

```
terraform plan    # ~5 new resources (ACR, AKS, 2 role assignments)
terraform apply   # ~10 min - AKS provisioning
```

15.4 12.4 Get cluster access

```
az aks get-credentials -g gskplat-rg-platform -n gskplat-aks-shared --overwrite-existing
kubectl get nodes
```

You should see your node(s) Ready.

Chapter 16

Chapter 13: Iteration 3 — Platform Components

Three things go on the cluster: Istio (service mesh), ArgoCD (GitOps), Jenkins (CI).

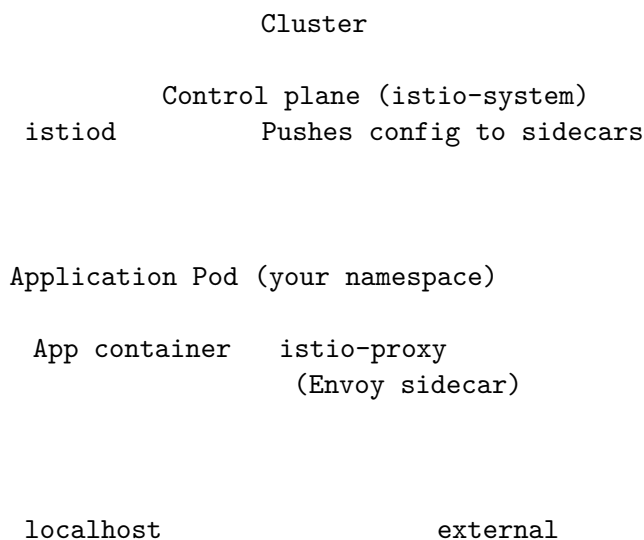
16.1 13.1 Istio

16.1.1 Why a service mesh?

Service mesh adds, transparent to your app: - **mTLS** between services (encryption + identity) - **Observability** (request rate, latency, error rate as metrics) - **Traffic management** (routing, retries, timeouts, circuit breaking) - **Canary deployments** (weighted traffic to versions)

Without a mesh, you'd implement each of these in your app. Same goes for every microservice. Painful.

16.1.2 The Istio architecture



istio-ingress (gateway) - public LB

Every Pod with `istio-injection: enabled` namespace label gets an `istio-proxy` sidecar injected.
The sidecar: - Intercepts ALL traffic in/out of the Pod (using iptables rules) - Encrypts with mTLS
- Exports metrics to Prometheus - Applies routing rules from VirtualServices

16.1.3 Install Istio (Helm)

```
helm repo add istio https://istio-release.storage.googleapis.com/charts
helm repo update istio
```

```
helm install istio-base istio/base -n istio-system --create-namespace --wait
helm install istiod istio/istiod -n istio-system --values kubernetes/istio/istiod-values.yaml
helm install istio-ingress istio/gateway -n istio-system --values kubernetes/istio/gateway-values.yaml
```

Three Helm releases: - `istio-base` — CRDs - `istiod` — control plane - `istio-ingress` — public-facing gateway

16.1.4 Gateway and VirtualService

A **Gateway** describes a load balancer at the edge of the mesh:

```
apiVersion: networking.istio.io/v1
kind: Gateway
metadata:
  name: three-tier-gateway
  namespace: istio-system
spec:
  selector:
    istio: ingress      # matches the ingress-gateway pod
  servers:
    - port:
        number: 80
        name: http
        protocol: HTTP
      hosts:
        - "dev.gskplat.local"
        - "prod.gskplat.local"
        - "*"            # fallback
```

A **VirtualService** describes routing rules:

```
apiVersion: networking.istio.io/v1
kind: VirtualService
metadata:
  name: three-tier-dev
  namespace: dev
```

```
spec:
  hosts:
    - "dev.gskplat.local"
    - "dev.lvh.me"
  gateways:
    - istio-system/three-tier-gateway
  http:
    - match:
        - uri:
            prefix: /api/
        route:
          - destination:
              host: backend.dev.svc.cluster.local
              port:
                number: 5678
    - route:
        - destination:
            host: frontend.dev.svc.cluster.local
            port:
              number: 80
```

This routes: - dev.gskplat.local/api/* → backend service - dev.gskplat.local/* → frontend service

16.1.5 Sidecar tuning (the resource gotcha)

Default Istio sidecars request 100m CPU each. With many pods on a small cluster, scheduler runs out of CPU.

Add per-pod annotations:

```
metadata:
  annotations:
    sidecar.istio.io/proxyCPU: "10m"
    sidecar.istio.io/proxyMemory: "64Mi"
    sidecar.istio.io/proxyCPULimit: "100m"
    sidecar.istio.io/proxyMemoryLimit: "256Mi"
```

Or opt out completely (e.g., for stateful DBs):

```
metadata:
  annotations:
    sidecar.istio.io/inject: "false"
```

16.2 13.2 ArgoCD

16.2.1 Why GitOps?

CI doing `kubectl apply` works but loses these: - **Audit trail** — who changed cluster state, when, why? - **Source of truth** — Git or cluster? They diverge silently - **Rollback** — manual coordination - **Disaster recovery** — re-create cluster from Git

GitOps fixes this: - **Git is the source of truth** - A controller (ArgoCD) syncs cluster to match Git - Drift is detected and (optionally) auto-corrected - Every deploy is a reviewable commit

16.2.2 Install ArgoCD

```
helm repo add argo https://argoproj.github.io/argo-helm
helm install argocd argo/argo-cd -n argocd --create-namespace --values kubernetes/argocd/values.yaml
```

16.2.3 Application CRD

An ArgoCD Application says “watch this Git path, sync to this cluster path”:

```
apiVersion: argoproj.io/v1alpha1
kind: Application
metadata:
  name: three-tier-dev
  namespace: argocd
spec:
  source:
    repoURL: https://github.com/myorg/myrepo.git
    targetRevision: master
    path: kubernetes/apps/three-tier
    helm:
      valueFiles:
        - values-dev.yaml
  destination:
    server: https://kubernetes.default.svc
    namespace: dev
  syncPolicy:
    automated:
      prune: false
      selfHeal: true
```

`automated.selfHeal: true` = ArgoCD reverts any drift between cluster and Git. `prune: false` = ArgoCD won't delete resources (safety).

16.3 13.3 Jenkins

16.3.1 Why Jenkins (still)?

- Mature, widely understood
- Free, on-prem-friendly
- Huge plugin ecosystem
- Runs IN the cluster — direct kubeconfig access without exposing the API

16.3.2 Install Jenkins (Helm)

```
helm install jenkins jenkins/jenkins -n jenkins --create-namespace \
  --values kubernetes/jenkins/values.yaml --wait
```

Important values:

```
controller:
  resources:
    requests:
      cpu: 250m           # trimmed for small cluster
      memory: 2Gi
  persistence:
    size: 8Gi
    storageClass: managed-csi
```

16.3.3 Kubernetes plugin for dynamic agents

Instead of pre-provisioned agents, Jenkins spins up a Pod per build. Pod includes containers for each tool: - **node** for tests/lint - **kaniko** for Docker builds - **helm** for deploys - **trivy** for scans

```
pipeline {
  agent {
    kubernetes {
      yaml '''
      apiVersion: v1
      kind: Pod
      spec:
        containers:
          - name: node
            image: node:20-alpine
            command: [cat]
            tty: true
          - name: kaniko
            image: gcr.io/kaniko-project/executor:debug
            command: [cat]
            tty: true
      '''
    }
  }
  stages {
    stage('Test') {
      steps {
        container('node') {
          sh 'npm ci && npm test'
        }
      }
    }
    stage('Build') {
      steps {
        container('kaniko') {
          sh '/kaniko/executor --context=. --destination=myacr.azurecr.io/myapp:1.0'
        }
      }
    }
  }
}
```

```
}  
}  
}
```

After iter 3: three platform tools running.

Chapter 17

Chapter 14: Iteration 4 — Three-Tier App

17.1 14.1 The umbrella chart

```
kubernetes/apps/three-tier/  
  Chart.yaml          # umbrella, depends on subcharts  
  values.yaml         # defaults  
  values-dev.yaml     # dev overrides  
  values-prod.yaml    # prod overrides  
  charts/  
    frontend/  
      Chart.yaml  
      values.yaml  
      templates/  
        deployment.yaml  
        service.yaml  
    backend/  
    database/  
  istio/  
    gateway.yaml  
    virtualservice-dev.yaml  
    virtualservice-prod.yaml
```

17.2 14.2 Frontend subchart

```
# charts/frontend/templates/deployment.yaml  
apiVersion: apps/v1  
kind: Deployment  
metadata:  
  name: frontend  
spec:  
  replicas: {{ .Values.replicaCount }}
```

```

selector:
  matchLabels:
    app: frontend
template:
  metadata:
    labels:
      app: frontend
    annotations:
      sidecar.istio.io/proxyCPU: "10m"
  spec:
    containers:
      - name: nginx
        image: "{{ .Values.image.repository }}:{{ .Values.image.tag }}"
        ports:
          - containerPort: 80
        {{- if .Values.useConfigMapHTML }}
        volumeMounts:
          - name: html
            mountPath: /usr/share/nginx/html
        {{- end }}
        resources:
          {{- toYaml .Values.resources | nindent 12 }}
        {{- if .Values.useConfigMapHTML }}
      volumes:
        - name: html
          configMap:
            name: frontend-html
        {{- end }}

```

useConfigMapHTML toggle lets us switch between placeholder mode (nginx serving HTML from ConfigMap) and real-image mode (vite-built React app).

17.3 14.3 Database subchart (StatefulSet)

```

# charts/database/templates/statefulset.yaml
apiVersion: apps/v1
kind: StatefulSet
metadata:
  name: database
spec:
  serviceName: database
  replicas: 1
  selector:
    matchLabels:
      app: database
  template:
    metadata:
      labels:

```

```

    app: database
  annotations:
    sidecar.istio.io/inject: "false"  # opt out of mTLS for stateful pod
  spec:
    containers:
      - name: postgres
        image: "{{ .Values.image.repository }}:{{ .Values.image.tag }}"
        env:
          - name: POSTGRES_DB
            value: {{ .Values.database }}
          - name: POSTGRES_USER
            value: {{ .Values.username }}
          - name: POSTGRES_PASSWORD
            valueFrom:
              secretKeyRef:
                name: database-credentials
                key: password
        volumeMounts:
          - name: data
            mountPath: /var/lib/postgresql/data
    volumeClaimTemplates:
      - metadata:
          name: data
        spec:
          accessModes: [ReadWriteOnce]
          storageClassName: {{ .Values.storage.storageClass }}
          resources:
            requests:
              storage: {{ .Values.storage.size }}

```

volumeClaimTemplates creates a separate PVC per Pod ordinal. database-0 gets data-database-0.

17.4 14.4 Deploy

```
# Create namespaces with istio-injection labels
```

```
kubectl apply -f kubernetes/apps/namespaces/dev.yaml
```

```
kubectl apply -f kubernetes/apps/namespaces/prod.yaml
```

```
# Apply Istio Gateway
```

```
kubectl apply -f kubernetes/apps/three-tier/istio/gateway.yaml
```

```
# Deploy app
```

```
cd kubernetes/apps/three-tier
```

```
helm dependency update
```

```
helm upgrade --install three-tier . -n dev --values values-dev.yaml --wait
```

```
helm upgrade --install three-tier . -n prod --values values-prod.yaml --wait
```

```
# Apply VirtualServices
kubectl apply -f istio/virtualservice-dev.yaml
kubectl apply -f istio/virtualservice-prod.yaml
```

17.5 14.5 Test via port-forward

```
# Forward istio-ingress
kubectl port-forward -n istio-system svc/istio-ingress 9999:80 &
```

```
# Hit with Host header
curl -H "Host: dev.gskplat.local" http://localhost:9999/
curl -H "Host: prod.gskplat.local" http://localhost:9999/
```

Different greetings per env confirm namespace routing works.

Chapter 18

Chapter 15: Iteration 5 — Real Code + Jenkins Pipeline

18.1 15.1 Backend app (Node + Express)

```
// apps/backend/server.js
const express = require('express');
const app = express();

app.get('/health', (req, res) => res.status(200).send('ok'));

app.get('/api/info', (req, res) => {
  res.json({
    env: process.env.APP_ENV || 'unknown',
    version: process.env.APP_VERSION || 'unknown',
    namespace: process.env.POD_NAMESPACE,
    pod: process.env.HOSTNAME,
    timestamp: new Date().toISOString()
  });
});

const PORT = process.env.PORT || 5678;
app.listen(PORT, () => console.log(`Listening on ${PORT}`));

// apps/backend/server.test.js
const request = require('supertest');
const app = require('./server');

describe('backend', () => {
  it('GET /health returns 200', async () => {
    const res = await request(app).get('/health');
    expect(res.statusCode).toBe(200);
  });
});
```


18.2 15.2 Backend Dockerfile

```
FROM node:20-alpine3.22 AS deps
WORKDIR /app
COPY package.json package-lock.json ./
RUN npm ci --omit=dev --no-audit --no-fund

FROM node:20-alpine3.22 AS runtime
WORKDIR /app
USER node
COPY --chown=node:node --from=deps /app/node_modules ./node_modules
COPY --chown=node:node server.js ./

ENV NODE_ENV=production
ENV PORT=5678
EXPOSE 5678

CMD ["node", "server.js"]
```

18.3 15.3 Jenkinsfile — multi-container pipeline

```
pipeline {
  agent {
    kubernetes {
      yaml '''
apiVersion: v1
kind: Pod
spec:
  serviceAccountName: jenkins
  containers:
    - name: node-backend
      image: node:20-alpine3.22
      command: [cat]
      tty: true
    - name: kaniko-backend
      image: gcr.io/kaniko-project/executor:debug
      command: [cat]
      tty: true
      volumeMounts:
        - name: docker-config
          mountPath: /kaniko/.docker
    - name: helm
      image: alpine/k8s:1.30.6
      command: [cat]
      tty: true
    - name: trivy
      image: aquasec/trivy:0.58.0
```

```

        command: [cat]
        tty: true
    - name: gitleaks
      image: zricethezav/gitleaks:v8.21.0
      command: [cat]
      tty: true
volumes:
  - name: docker-config
    secret:
      secretName: acr-docker-config
      items:
        - key: .dockerconfigjson
          path: config.json
'''
    }
}

environment {
    ACR = 'gskplatacrn73d5y.azurecr.io'
    IMAGE = "${ACR}/three-tier/backend"
    TAG = "build-${env.BUILD_NUMBER}"
}

stages {
    stage('CI checks') {
        parallel {
            stage('Backend tests + lint') {
                steps {
                    container('node-backend') {
                        dir('apps/backend') {
                            sh 'npm ci && npm test && npm run lint'
                        }
                    }
                }
            }
            stage('Secret scan') {
                steps {
                    container('gitleaks') {
                        sh 'gitleaks detect --source=. --no-banner'
                    }
                }
            }
        }
    }

    stage('Build backend image') {
        steps {
            container('kaniko-backend') {

```

```

        sh """
            /kaniko/executor \\\
            --context=\$(pwd)/apps/backend \\\
            --destination=${IMAGE}:${TAG} \\\
            --destination=${IMAGE}:latest \\\
            --cache=true --cache-repo=${ACR}/cache
        """
    }
}

stage('Trivy scan') {
    steps {
        container('trivy') {
            sh "trivy image --severity HIGH,CRITICAL --ignore-unfixed ${IMAGE}:${TAG} || true"
        }
    }
}

stage('Deploy dev') {
    steps {
        container('helm') {
            sh """
                helm upgrade --install three-tier kubernetes/apps/three-tier \\\
                --namespace dev \\\
                --values kubernetes/apps/three-tier/values-dev.yaml \\\
                --set backend.image.repository=${IMAGE} \\\
                --set backend.image.tag=${TAG} \\\
                --set backend.useEchoArgs=false \\\
                --wait --timeout 5m
            """
        }
    }
}

stage('E2E tests on dev') {
    steps {
        container('helm') {
            sh '''
                kubect1 run e2e-${BUILD_NUMBER} --rm -i --restart=Never -n dev \\\
                --image=curlimages/curl --command -- \\\
                curl -fsS http://backend.dev.svc.cluster.local:5678/health
            '''
        }
    }
}

stage('Approve prod') {

```

```

    steps {
      timeout(time: 5, unit: 'MINUTES') {
        input message: 'Promote to prod?', ok: 'Promote'
      }
    }
  }

  stage('Deploy prod') {
    // ... same as dev with prod values ...
  }
}

```

18.4 15.4 What this pipeline does

1. Spins up an agent Pod with 5 tool containers
2. Runs tests + lint + secret scan in parallel
3. kaniko builds the image (no Docker daemon needed) and pushes to ACR
4. trivy scans for CVEs (report mode)
5. helm upgrades the dev environment
6. E2E test hits /health
7. Waits for human approval
8. helm upgrades prod

18.5 15.5 The ACR secret

kaniko needs ACR creds. Create a docker-registry secret:

```

kubectl create secret docker-registry acr-docker-config \
  --namespace jenkins \
  --docker-server=gskplatacrn73d5y.azurecr.io \
  --docker-username="$ARM_CLIENT_ID" \
  --docker-password="$ARM_CLIENT_SECRET"

```

This Secret is mounted into the kaniko container at /kaniko/.docker/config.json. kaniko auto-uses it for push.

18.6 15.6 ArgoCD applications (drift detection)

```

apiVersion: argoproj.io/v1alpha1
kind: Application
metadata:
  name: three-tier-dev
  namespace: argocd
spec:
  source:
    repoURL: https://github.com/myorg/myrepo.git
    targetRevision: master

```

```
path: kubernetes/apps/three-tier
helm:
  valueFiles: [values-dev.yaml]
destination:
  server: https://kubernetes.default.svc
  namespace: dev
syncPolicy:
  # NO automated block - drift-only
  syncOptions: [CreateNamespace=false]
```

Why drift-only in iter 5? Because Jenkins is the deployer. After every Jenkins deploy, ArgoCD reports `OutOfSync` because the image tag in cluster differs from Git. That's the signal.

In iter 6 we switch ArgoCD to auto-sync and have CI write image tags to Git → ArgoCD picks them up. True GitOps.

Chapter 19

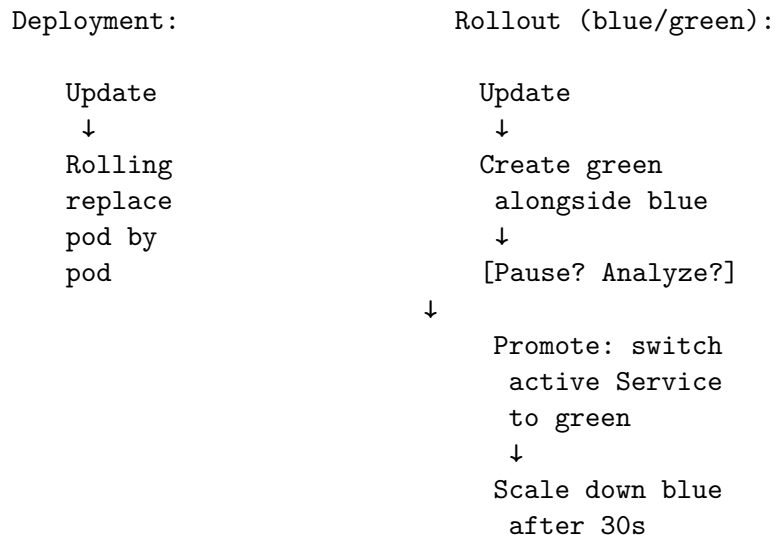
Chapter 16: Iteration 6 — GitHub Actions + Argo Rollouts + Prometheus

This is where the lab becomes production-grade. Three new components: GitHub Actions as alternative CI, Argo Rollouts for blue/green, Prometheus for analysis gates.

19.1 16.1 Argo Rollouts

19.1.1 What it is

A Kubernetes controller that replaces `Deployment` with a `Rollout` CRD. Rollout knows how to do: - **Blue/Green** — two `ReplicaSets`, swap at once - **Canary** — gradually shift traffic - **Analysis** — query metrics before promoting



19.1.2 Install

```
kubectl create namespace argo-rollouts
kubectl apply -n argo-rollouts -f https://github.com/argoproj/argo-rollouts/releases/latest/download/

# CLI plugin (run on your VM)
curl -LO https://github.com/argoproj/argo-rollouts/releases/latest/download/kubectl-argo-rollouts
chmod +x kubectl-argo-rollouts-linux-amd64
sudo mv kubectl-argo-rollouts-linux-amd64 /usr/local/bin/kubectl-argo-rollouts
```

19.1.3 Rollout spec

```
apiVersion: argoproj.io/v1alpha1
kind: Rollout
metadata:
  name: frontend
spec:
  replicas: 1
  strategy:
    blueGreen:
      activeService: frontend           # serves live traffic
      previewService: frontend-preview # serves test traffic for green
      autoPromotionEnabled: false      # manual gate
      scaleDownDelaySeconds: 30        # keep blue around for instant rollback
      prePromotionAnalysis:
        templates:
          - templateName: frontend-success-rate
        args:
          - name: service-name
            value: frontend-preview
  selector:
    matchLabels:
      app: frontend
  template:
    metadata:
      labels:
        app: frontend
    spec:
      containers:
        - name: nginx
          image: myacr.azurecr.io/frontend:sha-abc123
```

The Rollout creates two Services: - **frontend** — active, points to live ReplicaSet - **frontend-preview** — preview, points to new ReplicaSet during a deploy

19.1.4 Commands

```
# View
kubectl argo rollouts get rollout frontend -n dev
```

```

kubectl argo rollouts get rollout frontend -n dev --watch      # live updates

# Control
kubectl argo rollouts promote frontend -n dev                # green → active
kubectl argo rollouts abort frontend -n dev                  # kill green, keep blue
kubectl argo rollouts undo frontend -n dev                   # instant rollback
kubectl argo rollouts retry rollout frontend -n dev           # retry aborted

# Open the dashboard
kubectl argo rollouts dashboard --port 3100

```

19.2 16.2 Prometheus

19.2.1 What it is

Time-series database for metrics. Pulls (scrapes) metrics from configured endpoints.

```

      scrape
App pod      Prometheus
/metrics     every 15s  (stores time series)

```

```

      query via PromQL

      scrape
istio-proxy
/stats/prom

```

```

Grafana dashboards
AnalysisTemplate
Alertmanager

```

19.2.2 Install via kube-prometheus-stack

```

helm repo add prometheus-community https://prometheus-community.github.io/helm-charts
helm install prometheus prometheus-community/kube-prometheus-stack \
  --namespace monitoring --create-namespace \
  --set grafana.adminPassword=admin \
  --wait

```

This installs: - Prometheus + operator - Grafana - node-exporter (node metrics) - kube-state-metrics (Kubernetes object metrics)

19.2.3 PodMonitor (CRD that creates scrape config)

```

apiVersion: monitoring.coreos.com/v1
kind: PodMonitor

```



```

metadata:
  name: envoy-stats-monitor
  namespace: monitoring
  labels:
    release: prometheus          # match prometheus operator's labelSelector
spec:
  selector:
    matchExpressions:
      - { key: istio-prometheus-ignore, operator: DoesNotExist }
  namespaceSelector:
    any: true
  podMetricsEndpoints:
    - path: /stats/prometheus
      port: http-envoy-prom
      interval: 15s
      relabelings:
        - sourceLabels: [__meta_kubernetes_pod_container_name]
          action: keep
          regex: "istio-proxy"

```

Now Prometheus scrapes every istio-proxy sidecar. `istio_requests_total` shows up.

19.2.4 PromQL basics

```

# Total requests in last 5 minutes
sum(rate(istio_requests_total[5m]))

# Group by service
sum by (destination_service_name) (rate(istio_requests_total[5m]))

# Success rate
sum(rate(istio_requests_total{response_code!~"5.."}[2m]))
/
sum(rate(istio_requests_total[2m]))

# 99th percentile latency
histogram_quantile(0.99, sum(rate(istio_request_duration_milliseconds_bucket[5m])) by (le))

```

19.3 16.3 AnalysisTemplate

Now we can wire Prometheus to Argo Rollouts:

```

apiVersion: argoproj.io/v1alpha1
kind: AnalysisTemplate
metadata:
  name: frontend-success-rate
  namespace: dev
spec:
  args:

```

```

- name: service-name
metrics:
  # Gate 1: at least 1 req/sec on preview
  - name: min-traffic
    successCondition: result[0] >= 1
    failureLimit: 3
    interval: 30s
    count: 3
    provider:
      prometheus:
        address: http://prometheus-kube-prometheus-prometheus.monitoring.svc:9090
        query: |
          sum(rate(istio_requests_total{
            reporter="destination",
            destination_service_name="{{args.service-name}}"
          }[1m]))
          or vector(0)

  # Gate 2: success rate >= 95%
  - name: success-rate
    successCondition: result[0] >= 0.95
    failureLimit: 3
    interval: 30s
    count: 3
    provider:
      prometheus:
        address: http://prometheus-kube-prometheus-prometheus.monitoring.svc:9090
        query: |
          (
            sum(rate(istio_requests_total{
              reporter="destination",
              destination_service_name="{{args.service-name}}",
              response_code!~"5.."
            }[2m]))
            /
            sum(rate(istio_requests_total{
              reporter="destination",
              destination_service_name="{{args.service-name}}"
            }[2m]))
          )
          or vector(1)

```

19.3.1 Two-gate analysis explained

Both gates must pass. If either fails 3 times in a row, the rollout aborts.

Why min-traffic? Without it, the Prometheus query for success rate returns empty `result[]` for a zero-traffic preview Service. Then `result[0]` crashes the analyzer with `reflect: slice index`

out of range. We add `or vector(1)` to handle empty — but that makes zero-traffic = 100% success = false positive.

`min-traffic` gate forces traffic to flow before success-rate analysis runs. Without traffic, the gate fails, rollout aborts.

19.3.2 Flow

Argo Rollouts creates green ReplicaSet

Pre-promotion analysis fires

Query min-traffic (1 req/sec on preview Service)

Operator/CI must drive traffic to frontend-preview

Pass (3× check)? continue

Fail (3× check)? abort

Query success-rate (% non-5xx)

Pass (3× check)? continue

Fail (3× check)? abort

Rollout pauses at BlueGreenPause

Auto-promote (if `autoPromotionEnabled: true`)

OR

Manual: `kubectl argo rollouts promote`

Switch active Service to green ReplicaSet

After `scaleDownDelaySeconds=30`, blue is scaled down

19.4 16.4 GitHub Actions workflow

`name:` Three-Tier CI/CD

`on:`

`workflow_dispatch:` *# manual trigger only*

```

    inputs:
      deploy_env:
        type: choice
        options: [dev, prod]

permissions:
  contents: write          # so GITHUB_TOKEN can push commits

env:
  ACR_LOGIN_SERVER: myacr.azurecr.io
  BACKEND_IMAGE: myacr.azurecr.io/three-tier/backend
  FRONTEND_IMAGE: myacr.azurecr.io/three-tier/frontend
  CHART_PATH: kubernetes/apps/three-tier

jobs:
  # 1. Compute git SHA tag - done ONCE, shared with all jobs
  resolve-tag:
    runs-on: ubuntu-latest
    outputs:
      tag: ${ steps.tag.outputs.tag }
    steps:
      - uses: actions/checkout@v4
      - id: tag
        run: echo "tag=sha-$(git rev-parse --short=7 HEAD)" >> "$GITHUB_OUTPUT"

  # 2. CI checks - parallel
  backend-ci:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with:
          node-version: "20"
          cache: npm
          cache-dependency-path: apps/backend/package-lock.json
      - run: npm ci && npm test && npm run lint
        working-directory: apps/backend

  # ... frontend-ci, secret-scan ...

  # 3. Build images - uses SAME tag
  build-backend:
    needs: [backend-ci, secret-scan, resolve-tag]
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - name: Enable containerd image store
        run: |

```

```

        echo '{"features": {"containerd-snapshotter": true}}' | sudo tee /etc/docker/daemon.json
        sudo systemctl restart docker
- uses: azure/docker-login@v2
  with:
    login-server: ${ env.ACR_LOGIN_SERVER }
    username: ${ secrets.ACR_CLIENT_ID }
    password: ${ secrets.ACR_CLIENT_SECRET }
- uses: docker/build-push-action@v6
  with:
    context: apps/backend
    push: true
    tags: |
      ${ env.BACKEND_IMAGE }:${ needs.resolve-tag.outputs.tag }
      ${ env.BACKEND_IMAGE }:latest
    cache-from: type=gha,scope=backend
    cache-to: type=gha,scope=backend,mode=max

# 4. Trivy scan
trivy-scan:
  needs: [build-backend, build-frontend, resolve-tag]
  strategy:
    matrix:
      image: [backend, frontend]
  runs-on: ubuntu-latest
  steps:
    - uses: actions/checkout@v4
    - uses: aquasecurity/trivy-action@master
      with:
        image-ref: ${ env.ACR_LOGIN_SERVER }/three-tier/${ matrix.image }:${ needs.resolve-tag.outputs.tag }
        severity: HIGH,CRITICAL
        ignore-unfixed: true
        exit-code: "0" # report only
    - uses: actions/upload-artifact@v4
      with:
        name: trivy-${ matrix.image }-report
        path: trivy-${ matrix.image }.txt

# 5. Commit dev image tag to Git
update-dev-tags:
  needs: [trivy-scan, resolve-tag]
  environment: dev
  runs-on: ubuntu-latest
  steps:
    - uses: actions/checkout@v4
      with:
        token: ${ secrets.GITHUB_TOKEN }
    - name: Update values-dev.yaml
      env:

```

```

    IMAGE_TAG: ${ needs.resolve-tag.outputs.tag }}
run: |
    python3 -c "
    import os, yaml
    tag = os.environ['IMAGE_TAG']
    f = '${ env.CHART_PATH }}/values-dev.yaml'
    with open(f) as fp: v = yaml.safe_load(fp)
    v['frontend']['image']['tag'] = tag
    v['backend']['image']['tag'] = tag
    v['backend']['env']['APP_VERSION'] = tag
    with open(f, 'w') as fp: yaml.safe_dump(v, fp, default_flow_style=False, sort_keys=False)
    "
- name: Commit and push
  env:
    IMAGE_TAG: ${ needs.resolve-tag.outputs.tag }}
  run: |
    git config user.name "github-actions[bot]"
    git config user.email "github-actions[bot]@users.noreply.github.com"
    git add ${ env.CHART_PATH }}/values-dev.yaml
    git commit -m "ci(dev): bump images to ${IMAGE_TAG}" || echo "no changes"
    git pull --rebase origin master
    git push origin master

# 6. Manual gate for prod
approve-prod:
  needs: [update-dev-tags]
  environment: prod # GitHub Environment with required reviewers
  runs-on: ubuntu-latest
  steps:
    - run: echo "Approved for prod"

# 7. Commit prod tag - SAME SHA as dev
update-prod-tags:
  needs: [approve-prod, resolve-tag]
  runs-on: ubuntu-latest
  # ... similar to update-dev-tags but values-prod.yaml ...

```

19.5 16.5 What makes this production-grade

1. **Git SHA as image tag** — immutable, traceable to commit
2. **Image promotion** — same SHA flows dev → prod (build once, deploy many)
3. **Auto-promote in dev** — fast developer feedback
4. **Manual gate in prod** — GitHub Environment with required reviewers
5. **No kubeconfig in CI** — CI just commits to Git, ArgoCD applies
6. **Min-traffic gate** — prevents false-positive promotions
7. **Trivy report-only** — non-blocking but archived for audit
8. **Concurrency** — parallel CI jobs save wallclock time

19.6 16.6 The full flow visualized

Developer pushes commit (or manually triggers workflow)

GitHub Actions

```
resolve-tag      → sha-abc1234
  backend-ci
  frontend-ci    parallel    build-backend
  secret-scan                    build-frontend
                                trivy-scan
                                update-dev-tags
                                (commits sha-abc1234
                                 to values-dev.yaml,
                                 pushes to master)
```

ArgoCD detects new commit on master

applies updated Helm chart to dev namespace
Rollout spec gets new image tag sha-abc1234

Argo Rollouts

1. Creates green ReplicaSet with new image
2. Routes frontend-preview Service to green
3. Runs prePromotionAnalysis
 - min-traffic gate (must see 1 req/s on green)
 - success-rate gate (95% non-5xx)
4. If pass: auto-promote (dev) or pause (prod)
 - Active Service flips to green
 - Blue scaled down after 30s grace period
5. If fail: auto-abort
 - Green scaled to 0, blue keeps serving

GitHub Actions resumes

```
approve-prod (paused for reviewer click)
↓ after approval
update-prod-tags (commits SAME sha-abc1234 to values-prod)
```

↓
ArgoCD syncs prod, same flow as dev but autoPromote=false
↓
Operator: `kubectl argo rollouts promote frontend -n prod`

Chapter 20

Chapter 17: Iteration 7 — CircleCI (the third CI for portfolio)

20.1 17.1 Why a third CI?

For a Platform Engineer interview, showing you can speak to multiple CI systems is valuable. Jenkins is old-school in-house, GitHub Actions is the modern GitHub-integrated option, CircleCI is the multi-tenant SaaS with strong orb ecosystem.

20.2 17.2 CircleCI mental model

Pipeline (triggered by Git push or manual)
 Workflow (orchestrates jobs in a DAG)
 Job (runs in an executor)
 Steps (commands)

CircleCI term	GitHub Actions equivalent
Pipeline	Workflow
Workflow	Workflow
Job	Job
Executor	<code>runs-on</code> : runner
Orb	Action
Context	Environment
Step	Step

20.3 17.3 CircleCI configuration

```
version: 2.1
```

```
orbs:
```

```
  node: circleci/node@5.2.0
```

```

parameters:
  deploy_env:
    type: enum
    enum: ["dev", "prod"]
    default: "dev"

commands:
  install_and_cache_npm:
    parameters:
      app_dir:
        type: string
    steps:
      - restore_cache:
          keys:
            - v1-<< parameters.app_dir >>-deps-{{ checksum "apps/<< parameters.app_dir >>/package-lock.json" }}
            - v1-<< parameters.app_dir >>-deps-
      - run:
          name: npm ci
          working_directory: apps/<< parameters.app_dir >>
          command: npm ci --no-audit --no-fund
      - save_cache:
          key: v1-<< parameters.app_dir >>-deps-{{ checksum "apps/<< parameters.app_dir >>/package-lock.json" }}
          paths:
            - apps/<< parameters.app_dir >>/node_modules

jobs:
  resolve-tag:
    docker:
      - image: cimg/base:current
    steps:
      - checkout
      - run:
          name: Compute short SHA
          command: |
            mkdir -p workspace
            echo "sha-$(git rev-parse --short=7 HEAD)" > workspace/image_tag
      - persist_to_workspace:
          root: workspace
          paths: [image_tag]

  backend-ci:
    docker:
      - image: cimg/node:20.10
    steps:
      - checkout
      - install_and_cache_npm:
          app_dir: backend
      - run:

```

```

        name: Test
        working_directory: apps/backend
        command: npm test
    - run:
        name: Lint
        working_directory: apps/backend
        command: npm run lint

build-image:
  machine:
    image: ubuntu-2204:current
  parameters:
    app:
      type: enum
      enum: ["backend", "frontend"]
  steps:
    - checkout
    - attach_workspace:
        at: workspace
    - run:
        name: Login to ACR
        command: |
          AUTH=$(echo -n "$ACR_CLIENT_ID:$ACR_CLIENT_SECRET" | base64 -w0)
          mkdir -p ~/.docker
          printf '{"auths":{"%s":{"auth":"%s"}}}' "$ACR_LOGIN_SERVER" "$AUTH" > ~/.docker/cred
    - run:
        name: Build and push
        command: |
          IMAGE_TAG=$(cat workspace/image_tag)
          IMAGE="$ACR_LOGIN_SERVER/three-tier/<< parameters.app >>"
          docker build --tag "$IMAGE:$IMAGE_TAG" --tag "$IMAGE:latest" apps/<< parameters.ap
          docker push "$IMAGE:$IMAGE_TAG"
          docker push "$IMAGE:latest"

workflows:
  ci-cd:
    jobs:
      - resolve-tag
      - backend-ci
      - frontend-ci
      - secret-scan

    - build-image:
        name: build-backend
        app: backend
        context: azure-lab
        requires: [backend-ci, secret-scan, resolve-tag]

```

```

- build-image:
  name: build-frontend
  app: frontend
  context: azure-lab
  requires: [frontend-ci, secret-scan, resolve-tag]

- hold-prod:
  type: approval
  requires: [update-dev-tags]

- update-prod-tags:
  context: github-write
  requires: [hold-prod]

```

20.4 17.4 CircleCI-specific concepts

20.4.1 Executors

```

# Docker (default, fast)
docker:
  - image: cimg/node:20.10

# Machine (full VM, supports Docker-in-Docker)
machine:
  image: ubuntu-2204:current

# macOS (for iOS builds)
macos:
  xcode: "15.0.0"

# Windows
machine:
  image: windows-server-2022-gui:current
  resource_class: windows.medium

```

Docker executor can't run `docker build` natively. Use **machine** for Docker builds.

20.4.2 Orbs

Versioned reusable packages:

```

orbs:
  node: circleci/node@5.2.0
  aws-cli: circleci/aws-cli@4.0
  kubernetes: circleci/kubernetes@1.3
  helm: circleci/helm@2.0

```

Use them in jobs:

```
jobs:
```

```
test:
  executor: node/default
  steps:
    - checkout
    - node/install-packages
    - run: npm test
```

20.4.3 Contexts

Org-level secrets (vs project-level env vars):

```
jobs:
  - deploy:
      context: production-secrets  # imports env vars from Context
```

Contexts can have role-based access (only DevOps team can use prod-secrets context). Big advantage over GitHub Actions secrets which are repo-scoped.

20.4.4 Test parallelism

```
jobs:
  test:
    parallelism: 4
    steps:
      - run:
          command: |
            TESTS=$(circleci tests glob "**/*.test.js" | circleci tests split --split-by=timings)
            jest $TESTS
```

`circleci tests split --split-by=timings` distributes test files across 4 containers, balancing by historical run time. Much harder in GitHub Actions.

20.4.5 Workspaces (sharing files between jobs)

```
- persist_to_workspace:
    root: workspace
    paths: [image_tag]

# In a later job:
- attach_workspace:
    at: workspace
- run: cat workspace/image_tag
```

Like GitHub Actions artifacts but faster (in-memory).

20.4.6 Manual approval

```
workflows:
  deploy:
    jobs:
      - test
```

```

- build:
  requires: [test]
- hold-prod:
  type: approval          # ← pauses, awaits human click
  requires: [build]
- deploy-prod:
  requires: [hold-prod]

```

type: approval job doesn't run anything — just pauses for a UI click.

20.5 17.5 CircleCI vs GitHub Actions comparison

Feature	GitHub Actions	CircleCI
Free tier	2000 min/month (private)	6000 min/month (Linux)
Integration with GitHub	Native	Via app install
Self-hosted runners	Yes	Yes (Server)
Orb / Action ecosystem	Marketplace (huge)	Orb registry (curated)
Approval gates	Environment reviewers	type: approval
Test parallelism	Manual matrix	Native circleci tests split
Secrets	Repo-scoped (or env)	Org Contexts with RBAC
Docker layer caching	GHA cache type=gha	Premium DLC
Windows/macOS	Yes (paid more)	Yes (paid more)
Pricing	Per minute	Per credit (~per minute)

20.6 17.6 Common gotchas (here doc, etc.)

CircleCI uses << for parameter substitution. This conflicts with bash here doc syntax:

```

# BAD - `<<` interpreted as CircleCI parameter
run: |
  cat > file <<EOF
  content
  EOF

```

```

# GOOD - use printf or escape
run: |
  printf 'content' > file

```

This bit our lab pipeline. The fix: replace here docs with `printf` or single-command python.

Chapter 21

Part III — Advanced Topics

Chapter 22

Chapter 18: Observability

22.1 18.1 The three pillars

Metrics	Logs	Traces
Aggregated numbers over time	Discrete events	Request paths across services
e.g., req/sec CPU % latency p99	e.g., "user X logged in"	e.g., client → API → DB → cache → response
Tool: Prometheus Datadog	Tool: Loki / ELK Splunk	Tool: Jaeger / Tempo Datadog APM

Our lab has metrics (Prometheus). Logs come for free from Container Insights (Azure side). Traces would be next.

22.2 18.2 Metrics with Prometheus

22.2.1 What we measure

Category	Example metric
RED method (request/error/duration)	istio_requests_total, istio_request_duration_milliseconds_bucket
USE method (utilization/saturation/errors)	container_cpu_usage_seconds_total, container_memory_usage_bytes
Business KPIs	orders_placed_total, signups_completed_total

Category	Example metric
Internal app metrics	db_queries_total, cache_hits_total

22.2.2 Exposition format

Prometheus pulls a text endpoint:

```
# HELP http_requests_total Total HTTP requests
# TYPE http_requests_total counter
http_requests_total{method="GET",status="200"} 1234
http_requests_total{method="GET",status="500"} 5

# HELP http_request_duration_seconds Request duration
# TYPE http_request_duration_seconds histogram
http_request_duration_seconds_bucket{le="0.005"} 1230
http_request_duration_seconds_bucket{le="0.01"} 1232
http_request_duration_seconds_bucket{le="+Inf"} 1234
```

Apps expose `/metrics`. Istio sidecars expose at port 15090.

22.2.3 Cardinality matters

Each unique label combination = one time series. Too many = Prometheus OOM:

```
BAD: http_requests_total{user_id="abc123"}      ← unbounded
GOOD: http_requests_total{endpoint="/api/info"} ← finite set
```

Don't put user IDs, request IDs, or timestamps as labels.

22.3 18.3 Logs

Centralized logging answers “what happened?” Tools: - **ELK** (Elasticsearch + Logstash + Kibana) - **EFK** (Elasticsearch + Fluentd + Kibana) - **Loki** (lightweight, label-based like Prometheus) - **Cloud-managed** — Azure Log Analytics, AWS CloudWatch, GCP Cloud Logging

In Kubernetes, the pattern:

App pod → stdout → kubelet → log driver → forwarder → backend

Forwarders: Fluentd, Fluent Bit, Promtail, Vector.

22.3.1 Structured logging

Bad:

```
INFO: User abc123 logged in from 1.2.3.4
```

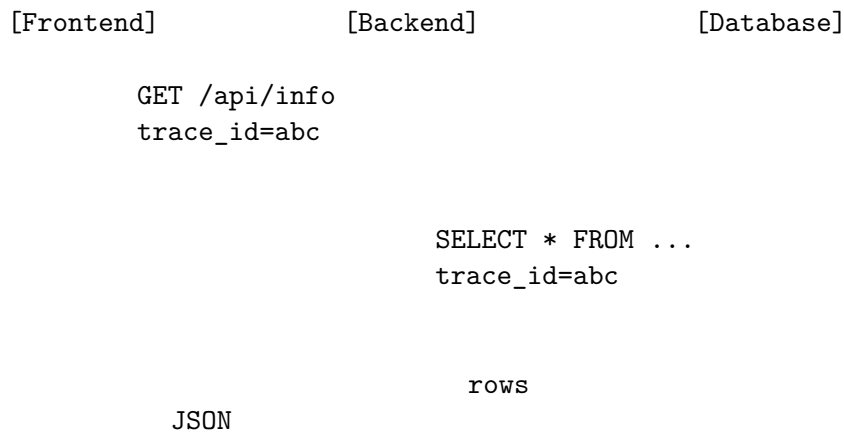
Good (JSON):

```
{"level":"info","msg":"login","user_id":"abc123","ip":"1.2.3.4","ts":"2026-05-14T12:00:00Z"}
```

Queryable. Filterable. Aggregatable.

22.4 18.4 Traces

For distributed systems. Each request gets a `trace_id` that follows it across services.



The trace shows: frontend → backend took 50ms total, of which DB call was 30ms.

Tools: Jaeger, Zipkin, Tempo, Datadog APM, NewRelic.

Istio auto-emits traces with `trace_id` propagated via B3 or W3C headers. App must propagate the header in calls.

22.5 18.5 SLI, SLO, SLA

Term	Definition	Example
SLI (Service Level Indicator)	A metric you measure	“% of requests that succeed in <200ms”
SLO (Service Level Objective)	An internal target	“99.9% of requests succeed in <200ms over a 28-day window”
SLA (Service Level Agreement)	A contract with consequences	“Customer gets credit if SLO not met for 1 month”

Burn rate alerting: alert when SLO budget is consumed faster than expected.

SLO: 99.9% over 28 days

Allowed errors: 0.1% × total requests

Budget: 0.1% × 28 days = ~40 minutes of full downtime per month

If we burned 50% of the budget in the first 2 days → fast burn → page on-call

22.6 18.6 Alerting

Prometheus alerts:

```
groups:
- name: api
```

```
rules:
- alert: HighErrorRate
  expr: |
    sum(rate(http_requests_total{status=~"5.."}[5m])) /
    sum(rate(http_requests_total[5m])) > 0.05
  for: 5m
  annotations:
    summary: "5xx error rate > 5% for 5 minutes"
    runbook: https://wiki.example.com/runbooks/high-error
```

Alertmanager → PagerDuty/Opsgenie/Slack.

22.6.1 Alert hygiene

- **Page only on customer-impacting issues** — don't page on disk warnings
 - **Every alert has a runbook** — “the on-call doesn't have to remember”
 - **No alert without an SLO** — otherwise you're alerting on arbitrary thresholds
 - **Burn rate over single events** — flapping noise vs sustained issues
-

Chapter 23

Chapter 19: Security

23.1 19.1 Defense in depth

No single control protects you. Layer security:

1. Identity
SSO, MFA, least-privilege RBAC, Workload Identity
2. Network
Private clusters, NSGs, NetworkPolicies, WAF
3. Supply chain
Image scanning (Trivy), signing (Cosign), SBOM (Syft)
4. Workload
Non-root, drop caps, read-only rootfs, PSS restricted
5. Runtime
Falco for anomaly detection
6. Secrets
External Secrets, Key Vault, rotation
7. Application
SAST (SonarQube), DAST (ZAP), dependency scanning
8. Data
Encryption at rest, in transit, key management

23.2 19.2 Secrets management

23.2.1 Bad

- Hardcoded in code
- Hardcoded in values.yaml (our lab — known gap)
- Environment variables in Dockerfile

23.2.2 OK

- Kubernetes Secrets (base64, not encrypted; encryption-at-rest must be enabled)
- HashiCorp Vault (with auth and access control)

23.2.3 Best

- External Secrets Operator + Key Vault / AWS Secrets Manager
- Workload Identity for service-to-service auth (no shared secrets)
- Short-lived tokens, rotated automatically

23.3 19.3 Container security

Practice	Detail
Run as non-root	USER 1001 in Dockerfile; <code>runAsNonRoot: true</code> in PodSecurityContext
Drop capabilities	<code>securityContext.capabilities.drop: [ALL]</code>
Read-only rootfs	<code>readOnlyRootFilesystem: true</code>
No privileged	<code>privileged: false</code> (default but verify)
Minimal base	distroless, alpine, scratch
Pin by digest	<code>myimage@sha256:abc...</code> not <code>myimage:latest</code>
Scan images	Trivy, Grype, Snyk in CI
Sign images	Cosign + Sigstore
Admission policy	Kyverno/Gatekeeper enforces signing + scanning

23.4 19.4 Network policies

Default deny + explicit allow:

```
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: default-deny-all
  namespace: dev
spec:
  podSelector: {}
  policyTypes: [Ingress, Egress]
```

```

---
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: allow-frontend-to-backend
  namespace: dev
spec:
  podSelector:
    matchLabels:
      app: backend
  policyTypes: [Ingress]
  ingress:
    - from:
        - podSelector:
            matchLabels:
              app: frontend
      ports:
        - protocol: TCP
          port: 5678

```

Without these, pods can talk to anything. Kubernetes default is “allow all”.

23.5 19.5 Pod Security Standards (PSS)

Replaced PSPs (deprecated). Three profiles:

Profile	What it allows
privileged	Everything (default in many clusters)
baseline	No privileged escalation, hostNetwork, hostPID
restricted	Strict — non-root, drop caps, read-only rootfs, etc.

Apply via namespace label:

```

metadata:
  labels:
    pod-security.kubernetes.io/enforce: restricted

```

23.6 19.6 RBAC principles

- **Least privilege** — only what’s needed
- **Namespace-scoped > Cluster-scoped** — Role + RoleBinding before ClusterRole + ClusterRoleBinding
- **Service Accounts** — every Pod should have its own; never use **default**
- **Audit logging** — Kubernetes API audit logs to LAW/CloudWatch
- **Periodic review** — quarterly access review

Chapter 24

Chapter 20: Production-Grade Gaps in Our Lab

The lab is a learning artifact. For production, address these:

24.1 20.1 Identity & Secrets

Gap	Production fix
SP password for ACR push	Workload Identity — AKS pod assumes Azure AD app via OIDC federation
Hardcoded DB passwords in values.yaml	External Secrets Operator pulling from Azure Key Vault
Long-lived GitHub PAT	GitHub OIDC federation to Azure (no PAT needed)
Static SP credentials in ~/.azure-lab.env	Azure CLI with managed identity on a VM with system MSI

24.2 20.2 Network

Gap	Production fix
Public AKS API	Private cluster + bastion
No NetworkPolicies	Default deny + explicit allow per service
Istio PERMISSIVE mTLS	STRICT + PeerAuthentication
Public LoadBalancer	Internal LB behind App Gateway + WAF

24.3 20.3 Supply chain

Gap	Production fix
Image tag pinning	Digest pinning (@sha256:...)
Trivy report-only	Block on Critical; auto-PR fixes via Renovate
No image signing	Cosign sign in CI; Kyverno policy verifies at admission
No SBOM	Syft generates SPDX SBOM, stored as ACR artifact

24.4 20.4 Reliability

Gap	Production fix
Single AKS cluster	Multi-cluster fleet (active-passive across regions)
Postgres StatefulSet	Azure Database for PostgreSQL (managed, replicas, PITR)
No backups	Velero scheduled snapshots of cluster + PV state
No HPA	Horizontal Pod Autoscaler on CPU/memory + custom metrics
No automated rollback	Prometheus alert → webhook → <code>kubectl argo rollouts abort</code>

24.5 20.5 Process

Gap	Production fix
Direct push to master	Branch protection — PR required, CI must pass, 1 approval
Manual approval clicks	Change Advisory Board integration (ServiceNow)
No alerts	SLO-driven alerts in Alertmanager → PagerDuty
One environment per cluster	Separate staging cluster with prod-like traffic

Chapter 25

Chapter 21: Interview Questions and Answers

50 questions categorized. For each, the format is:

- **Q:** the question
- **Bad answer:** what NOT to say
- **Good answer:** what TO say
- **Why:** the reasoning

25.1 Kubernetes

25.1.1 1. Difference between a Deployment and a StatefulSet?

Bad: “Deployment is for apps, StatefulSet is for databases.”

Good: “Deployments treat pods as cattle — interchangeable, random naming, random delete order. StatefulSets treat pods as pets — stable identity (`mydb-0`, `mydb-1`), stable storage (each gets its own PVC), ordered start (0 before 1), ordered shutdown (reverse). Use Deployments for stateless apps. Use StatefulSets for anything that needs stable identity, stable storage, or ordered startup — typical examples are databases, message brokers, distributed consensus systems like Kafka or etcd.”

Why: Show the technical detail and the reasoning, not just the conclusion.

25.1.2 2. Walk through what happens when you run `kubectl apply -f deployment.yaml`

Good: “`kubectl` reads the YAML, validates it against API schema, sends it to the API server. API server authenticates the request (token, cert), authorizes via RBAC, persists to etcd. The Deployment controller (a control loop) sees the new Deployment, creates a ReplicaSet to match the spec. ReplicaSet controller sees the ReplicaSet has 0 of the desired pods, creates pod objects. Scheduler watches for unscheduled pods, finds a node with capacity, binds the pod. kubelet on that node sees the binding, pulls images, starts containers via container runtime (containerd). kubelet reports back to API server. Once ready, the Service’s endpoints get updated to include the pod IP, kube-proxy on each node updates iptables/IPVS rules to load balance to that pod.”

Why: Demonstrates understanding of the control plane architecture, control loops, and the role of each component.

25.1.3 3. How does a Service load balance to multiple Pods?

Good: “Service has a selector matching pod labels. The Endpoints (or EndpointSlices in newer versions) resource holds the list of pod IPs matching the selector — kept in sync by the endpoint controller. kube-proxy on each node watches Endpoints and programs iptables/IPVS rules: traffic to the Service ClusterIP gets DNAT’d to one of the pod IPs, chosen by hash. There’s no actual load balancer process — it’s pure kernel networking. The ‘load balancing’ is just N pod IPs sharing requests pseudo-randomly.”

Why: Many candidates think there’s a daemon doing the balancing. Showing you know it’s iptables/IPVS is depth.

25.1.4 4. Pod stuck in Pending. Debug steps.

Good:

1. `kubectl describe pod <name>` - events at the bottom usually tell you
2. Common: 'Insufficient cpu/memory' - check requests vs node allocatable
`kubectl describe nodes | grep -A 5 Allocatable`
3. 'FailedScheduling: 0/N nodes match' - node selectors, taints, affinity
4. 'PVC not bound' - StorageClass missing or PVC mistyped
5. 'No nodes available' - cluster empty (autoscaler off?)
6. `kubectl get events --sort-by='.lastTimestamp' | tail -20`

Why: Show a systematic debugging mindset.

25.1.5 5. What’s the role of a sidecar container?

Good: “Augments the main container without changing it. Common patterns: logging agent that ships logs from a shared volume, proxy that handles TLS termination or mesh networking (like Istio’s istio-proxy), monitoring agent that scrapes metrics, init container for one-time setup. Sidecars share the Pod’s network namespace and storage, so they can communicate via localhost or shared volumes.”

Why: Shows knowledge of the Pod-as-deployment-unit philosophy.

25.2 CI/CD

25.2.1 6. GitOps vs CIOps?

Good: “GitOps: Git is the source of truth for cluster state. A controller (ArgoCD or Flux) watches Git, applies changes to the cluster. CI’s job ends at committing to Git. CIOps: CI pipeline directly applies changes via `kubectl` or `helm`. Cluster state drifts from Git silently. Differences: GitOps has full audit trail in Git, easy rollback via `git revert`, automated drift detection. CIOps is simpler to set up but harder to scale across teams and clusters.”

25.2.2 7. Blue/green vs canary?

Good: “Both reduce blast radius. Blue/green: full stack of new version runs alongside old, instant cutover. Simpler — 100% traffic on one version. Use when: small risk tolerance, fast rollback critical, no traffic-shifting needed. Canary: gradually shift X% of traffic to new version, observe, increase. More nuanced — partial rollout. Use when: large user base where 5% sample is statistically meaningful, gradual confidence building, business KPIs need observation. Argo Rollouts supports both. Production teams often pick canary because it catches issues that only manifest at scale.”

25.2.3 8. How do you handle database migrations during deploys?

Good: “Decouple them. Three patterns: (1) Pre-deploy migration — Helm pre-upgrade hook runs a Job. Risky if migration is slow or fails. (2) Expand-contract — make the schema backward compatible. Step 1: add new column nullable, deploy app that writes both old and new. Step 2: backfill. Step 3: stop writing old. Step 4: deploy app that only reads new. Step 5: drop old column. (3) Online schema change tools — pt-online-schema-change for MySQL, similar for Postgres. We never block deploys on migrations in production.”

25.2.4 9. How do you decide between Helm and Kustomize?

Good: “Helm: when you need package management (install a community chart like Prometheus). When you need conditional logic or loops. Strong for shared libraries. Kustomize: when manifests are simple, you want pure YAML, environment variations are mostly patches. Built into kubectl. Less abstraction. Hybrid: many teams use Helm for third-party (Nginx Ingress, cert-manager) and Kustomize for in-house. Some use Helm with Kustomize for post-processing.”

25.2.5 10. Walk through a CI failure on a flaky test.

Good: “First, classify: is it a real bug intermittently exposed, or environmental? If real, fix the bug. If environmental — timing, network, shared state — fix the test. Common fixes: explicit waits instead of sleep, isolated databases per test (transaction rollback, schemaper, sanity), retries with backoff at the test level (not CI level — that hides real flakiness), pinning Docker image versions in test deps. If we can’t fix it now, mark the test skipped with a TODO and a Jira link. Never delete tests without root cause.”

25.3 Terraform

25.3.1 11. What’s a state file? Why remote backend?

Good: “Terraform’s state file maps your HCL resources to real cloud resource IDs. Required for diff computation. Remote backend (Azure Storage / S3) enables team collaboration — locking prevents simultaneous applies that would corrupt state. Local state is only safe for solo work. State contains sensitive data (passwords, keys), so the backend should be encrypted and access-controlled.”

25.3.2 12. What does terraform plan actually do?

Good: “Reads the current state file, queries the cloud APIs to refresh state (so it knows what really exists), compares against your HCL configuration, computes a diff. Output is a list of resources that will be created, changed, or destroyed. Doesn’t make any changes. Save the plan to a file

(`-out=tfplan`) and pass it to `apply` for guaranteed determinism — what was planned is exactly what gets applied.”

25.3.3 13. Two devs run `terraform apply` simultaneously.

Good: “The first one to start acquires a state lock (Azure blob lease, AWS DynamoDB lock, etc.). The second sees `Error acquiring state lock` and refuses to proceed. Without locking, one `apply` could overwrite the state changes of the other, corrupting state. After lock timeout (configurable), stale locks can be force-unlocked, but that’s a manual operation requiring confirmation.”

25.3.4 14. How do you import existing cloud resources into Terraform?

Good: “`terraform import <resource-address> <real-id>` adds the resource to state. Then you write the HCL to match. Run `plan` to see if your HCL matches reality — usually it doesn’t perfectly, so you adjust HCL until plan shows no changes. Newer Terraform has `import {}` blocks that combine both steps. For bulk imports, use `terraformer` or `aztfexport` tools. Don’t try to do an `apply` right after `import` — verify with `plan` first.”

25.4 Azure

25.4.1 15. AKS vs EKS vs GKE — when do you pick each?

Good: “Mostly business alignment. AKS for shops on Azure (Entra ID SSO, ExpressRoute on-prem, enterprise agreements). EKS for mature AWS shops with deep ecosystem integration. GKE for Google-aligned shops or teams that want autopilot mode (zero-node-management). Technically all are mature. AKS has best Workload Identity story. EKS has more compute options (Fargate, multiple AMIs). GKE invented Kubernetes and has historically led in autoscaling. The choice is rarely technical at this point.”

25.4.2 16. Why kubenet vs Azure CNI?

Good: “kubenet: pods get a non-routable IP, NAT through node IP for outbound. Simple, uses few VNet IPs (only nodes get VNet IPs). Pods can’t directly reach other Azure resources without NAT. Azure CNI: pods get VNet IPs, route directly to anything in the VNet (databases, App Service, etc.), but consumes a lot of subnet IPs (one per pod). Azure CNI Overlay is the modern compromise — pods get overlay IPs, nodes get VNet IPs, you still get the VNet routing for nodes.”

25.4.3 17. Service Principal vs Managed Identity?

Good: “Service Principal: an explicit Azure AD application identity with credentials (client secret or cert). Used by external systems (CI, on-prem tools). Long-lived credentials = risk. Managed Identity: Azure resources (VMs, AKS pods via Workload Identity) get an automatically-managed identity. No credentials to store. The cloud handles rotation. Always prefer Managed Identity when possible. SPs are a fallback for things outside Azure.”

25.5 Observability

25.5.1 18. RED method vs USE method?

Good: “Two complementary approaches. RED (Tom Wilkie) is request-centric: Rate, Errors, Duration. For service-level metrics — answers ‘is my service serving users well?’ USE (Brendan Gregg) is resource-centric: Utilization, Saturation, Errors. For infrastructure metrics — answers ‘is my CPU/disk/network healthy?’ In practice you measure both. RED for app health, USE for capacity planning.”

25.5.2 19. What’s an SLO?

Good: “Service Level Objective — an internal target measuring service quality from the user’s perspective. Example: 99.9% of HTTP requests succeed in <200ms over a rolling 28-day window. The 0.1% is your error budget. You burn it via real failures. Burn rate alerting (Google SRE book) pages on-call when budget burns too fast. SLO drives priorities — if you have budget, ship features. If you’ve burned it, focus on reliability work. SLA is the external contract built on top, with penalties for missing it.”

25.5.3 20. How does Prometheus collect metrics?

Good: “Prometheus pulls (scrapes). It hits HTTP endpoints (`/metrics` typically) and parses Prometheus exposition format. Service discovery is built in — Kubernetes, Consul, EC2 — Prometheus queries the cluster for what to scrape. Each scrape gets a timestamp, stored in TSDB (time-series database). Cardinality is critical: each unique label combination is a separate time series. High cardinality (request IDs, user IDs as labels) crashes Prometheus. Push gateway exists for short-lived jobs but is generally discouraged.”

25.6 Argo / GitOps specific

25.6.1 21. How does Argo Rollouts work under the hood?

Good: “It’s a custom controller (with a Rollout CRD) that replaces Deployments. Watches Rollout objects, manages ReplicaSets directly. Blue/green: when image changes, creates a new ReplicaSet (green), routes preview Service to green, runs pre-promotion analysis. Each AnalysisRun queries Prometheus per interval, checks the success condition. On pass, switches active Service selector to point at green. On fail, scales green down. The active/preview Services are pre-existing — Rollout just updates their selectors.”

25.6.2 22. Why might prePromotionAnalysis falsely pass?

Good: “Zero traffic. If no requests have hit the preview Service, your success rate query returns empty result, which crashes the analyzer. Common fix is `or vector(1)` so it returns 1.0 — but that means zero traffic = 100% success = false positive. The fix is a separate min-traffic gate that requires a minimum req/sec before the analysis runs. In production, this means CI must drive traffic to the preview (synthetic smoke tests, replayed prod traffic), not just sit and wait.”

25.6.3 23. How does ArgoCD detect drift?

Good: “ArgoCD periodically (every 3 min by default) renders the Helm chart or kustomize output from Git, then queries the cluster for the actual resources. Diffs them. If different, the Application shows OutOfSync. With `automated.selfHeal: true`, ArgoCD applies the Git state back to the cluster automatically. With `prune: false`, ArgoCD doesn’t delete resources from the cluster even if removed from Git (safety). Without `selfHeal`, drift is just reported, not corrected.”

25.6.4 24. ArgoCD shows Suspended. Is that bad?

Good: “Usually no — it means an Argo Rollouts Rollout is paused at a BlueGreenPause, waiting for a human to promote or for analysis to complete. That’s the production safety gate working as designed. Most dashboards alert only on Degraded, not Suspended. If you find Suspended noisy in your team, add an ArgoCD health override (Lua script) to map paused-Rollouts to Healthy. But honestly, learning what Suspended means is more useful than hiding it.”

Chapter 26

Part IV — Appendix

Chapter 27

Appendix A: Quick Reference Cheatsheets

27.1 Kubernetes daily commands

Context

```
kubectl config current-context
kubectl config use-context <name>
kubectl config get-contexts
```

Get

```
kubectl get pods -A
kubectl get pods -n <ns> -o wide
kubectl get pods -l app=frontend
kubectl get events --sort-by='.lastTimestamp' -n <ns>
```

Describe

```
kubectl describe pod <pod-name> -n <ns>
kubectl describe node <node-name>
```

Logs

```
kubectl logs <pod-name> -n <ns>
kubectl logs <pod-name> -c <container> --previous
kubectl logs -f <pod-name>
```

Exec

```
kubectl exec -it <pod-name> -- sh
kubectl exec <pod-name> -c <container> -- ls
```

Port-forward

```
kubectl port-forward -n <ns> svc/<svc-name> 8080:80
```

Apply

```
kubectl apply -f manifest.yaml
```



```
kubectl apply -k overlay/dev
```

Delete

```
kubectl delete -f manifest.yaml
```

```
kubectl delete pod <pod-name> --force --grace-period=0
```

Rollout

```
kubectl rollout status deployment/<name>
```

```
kubectl rollout history deployment/<name>
```

```
kubectl rollout undo deployment/<name>
```

Scale

```
kubectl scale deployment <name> --replicas=5
```

Debug

```
kubectl run -it --rm debug --image=busybox -- sh
```

```
kubectl debug node/<node> -it --image=busybox
```

27.2 Helm commands

```
helm repo add <name> <url>
```

```
helm repo update
```

```
helm search repo <term>
```

```
helm install <release> <chart>
```

```
helm install <release> <chart> --values values.yaml --set image.tag=v1
```

```
helm upgrade --install <release> <chart> --values values.yaml
```

```
helm list -A
```

```
helm history <release>
```

```
helm rollback <release> <revision>
```

```
helm uninstall <release>
```

```
helm template ./mychart # render without installing
```

27.3 Terraform commands

```
terraform init
```

```
terraform fmt
```

```
terraform validate
```

```
terraform plan -out=tfplan
```

```
terraform apply tfplan
```

```
terraform apply
```

```
terraform destroy
```

```
terraform output
```

```
terraform state list
```

```
terraform state show <resource>
```

```
terraform state rm <resource>
```

```
terraform import <resource> <id>
```

```
terraform workspace list
```

```
terraform workspace new dev
```

27.4 Argo Rollouts commands

```
kubectl argo rollouts get rollout <name> -n <ns>
kubectl argo rollouts get rollout <name> -n <ns> --watch
kubectl argo rollouts promote <name> -n <ns>
kubectl argo rollouts abort <name> -n <ns>
kubectl argo rollouts undo <name> -n <ns>
kubectl argo rollouts retry rollout <name> -n <ns>
kubectl argo rollouts dashboard --port 3100
```

27.5 Azure CLI

```
az login
az account list
az account set --subscription <id>
az group create -n myrg -l eastus2
az aks get-credentials -g myrg -n mycluster
az aks stop -g myrg -n mycluster
az aks start -g myrg -n mycluster
az acr login -n myacr
az acr repository list -n myacr
az ad sp create-for-rbac --name myapp --role Contributor --scopes /subscriptions/<sub>
az role assignment list --assignee <id> --all
```

27.6 Docker

```
docker build -t myapp:1.0 .
docker run -p 8080:80 myapp:1.0
docker run -d --name web -p 8080:80 myapp:1.0
docker ps
docker logs <container>
docker exec -it <container> sh
docker stop <container>
docker rm <container>
docker system prune -af
docker login myacr.azurecr.io
docker push myacr.azurecr.io/myapp:1.0
```

27.7 Git

```
git status
git add . && git commit -m "msg"
git pull --rebase origin master
git push origin <branch>
```

```
git checkout -b feature/x
git merge main
git rebase -i HEAD~3
git revert <commit>
git log --oneline --graph --all
git stash / git stash pop
```

Chapter 28

Appendix B: Building this PDF/DOCX

This handbook is markdown. To convert:

28.1 To PDF

Install pandoc + LaTeX

```
sudo apt install pandoc texlive-xetex texlive-fonts-extra
```

Convert

```
pandoc docs/textbook/platform-engineering-handbook.md \  
-o docs/textbook/platform-engineering-handbook.pdf \  
--pdf-engine=xelatex \  
--toc --toc-depth=3 \  
-V geometry:margin=1in \  
-V fontsize=11pt \  
--metadata-file=docs/textbook/platform-engineering-handbook.md
```

28.2 To DOCX

```
pandoc docs/textbook/platform-engineering-handbook.md \  
-o docs/textbook/platform-engineering-handbook.docx \  
--toc --toc-depth=3 \  
--reference-doc=docs/textbook/template.docx      # optional template
```

28.3 Quick install on Ubuntu

```
sudo apt update && sudo apt install -y pandoc texlive-xetex  
cd docs/textbook  
pandoc platform-engineering-handbook.md -o handbook.pdf --pdf-engine=xelatex --toc  
pandoc platform-engineering-handbook.md -o handbook.docx --toc
```

You now have both formats. Read on tablet or print.

Chapter 29

Appendix C: Where to Go Next

Topics worth your time after mastering this lab:

29.1 Advanced Kubernetes

- **Cluster API** — manage clusters as Kubernetes objects
- **Karpenter** (AWS) / **Cluster Autoscaler** — node autoscaling
- **KEDA** — event-driven autoscaling
- **OPA Gatekeeper** / **Kyverno** — policy as code
- **Crossplane** — provision cloud resources via Kubernetes CRDs

29.2 Advanced CI/CD

- **Backstage** — developer portal that wraps your platform
- **Tekton** — Kubernetes-native pipelines
- **Spinnaker** — multi-cloud canary deployment platform
- **Argo Workflows** — DAG-based job orchestration

29.3 Reliability

- **Velero** — backup and disaster recovery
- **Litmus** / **Chaos Mesh** — chaos engineering
- **Pyrra** / **Sloth** — SLO management
- **Robusta** / **Komodor** — Kubernetes incident response

29.4 Security

- **Trivy** / **Grype** / **Snyk** — vulnerability scanning (you have Trivy)
- **Cosign** / **Sigstore** — image signing
- **Falco** — runtime security
- **Tetragon** — eBPF-based security observability

29.5 Cost

- **OpenCost** / **Kubecost** — Kubernetes cost visibility
 - **Karpenter** — provisioning the cheapest node SKU
-

Chapter 30

End

This handbook covers the full journey from cloud fundamentals through a production-pattern AKS lab to advanced topics for further study.

The lab repo: <https://github.com/gkhandale-aziro/azure-platform-lab>

Re-read sections you struggle with. Run the commands. Break things on purpose. The work that wins interviews now is practicing the conversation, not building more.